

II užduotis

Vieno neurono mokymas sprendžiant
klasifikavimo uždavinį

II užduotis (Vieno neurono mokymas sprendžiant klasifikavimo uždavinį)

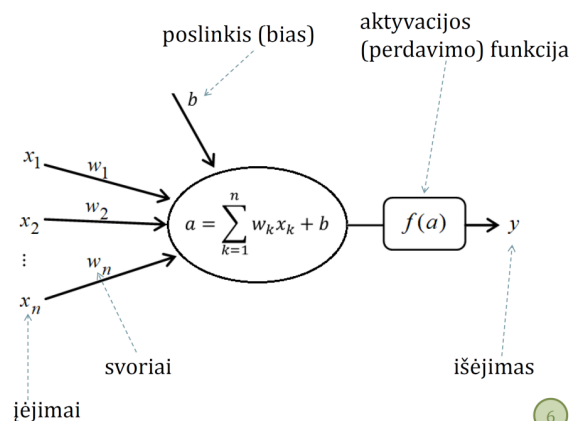
Užduoties tikslas – apmokyti vieną neuroną spręsti dviejų klasių uždavinį ir atlikti tyrimą.

Šioje užduotyje bus naudojama **krūties vėžio duomenų aibė**, kurią galima parsisiųsti iš <https://archive.ics.uci.edu/dataset/15/breast+cancer+wisconsin+original> (iš *.zip failo rinktis *breast-cancer-wisconsin.data*). Šiuose duomenyse yra dvi klasės: 2 – nepiktybinis navikas, 4 – piktybinis navikas.

Užduotyje reiks mokyti sigmoidinį neuroną. Sigmoidiniame neurone naudojama sigmoidinė aktyvacijos funkcija. Neurono išėjimams (*output, predicted class*) apskaičiuoti taip pat naudojama sigmoidinė funkcija. Neurono išėjimo reikšmės bus intervale (0; 1), tad norint nustatyti spėjamą klasę (*predicted class*), šias reikšmes reiks suapvalinti iki artimiausio sveiko skaičiaus (0 arba 1).

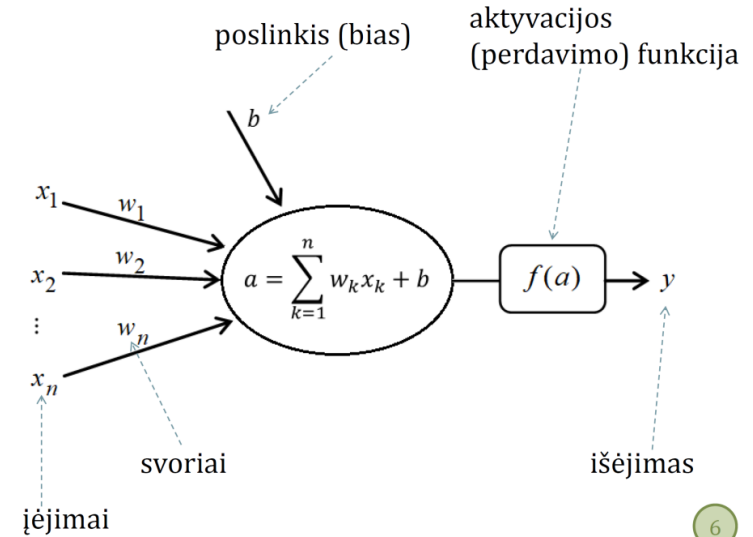
Sigmoidinio neurono mokymui reiks sukurti du algoritmus: (1) naudoti paketinį gradientinį nusileidimą, (2) naudoti stochastinį gradientinį nusileidimą

(žr. <https://emokymai.vu.lt/mod/resource/view.php?id=171951>, 43 ir 45 skaidrės).



Užduoties tikslas – apmokyti vieną neuroną spręsti dviejų klasių uždavinį ir atlikti tyrimą.

Šioje užduotyje bus naudojama **krūties vėžio duomenų aibė**, kurią galima parsisiųsti iš <https://archive.ics.uci.edu/dataset/15/breast+cancer+wisconsin+original> (iš *.zip failo rinktis *breast-cancer-wisconsin.data*). Šiuose duomenyse yra dvi klasės: 2 – nepiktybinis navikas , 4 – piktybinis navikas.





Breast Cancer Wisconsin (Original)

Donated on 7/14/1992

Original Wisconsin Breast Cancer Database

Dataset Characteristics

Multivariate

Subject Area

Life Science

Associated Tasks

Classification

Feature Type

Integer

Instances

699

Features

9

DOWNLOAD



IMPORT IN PYTHON

CITE

6 citations

62845 views

Keywords

health

cancer

Creators

William Wolberg

Dataset Information

Additional Information

Samples arrive periodically as Dr. Wolberg reports his clinical cases. The database therefore reflects this chronological grouping of the data. This grouping information appears immediately below, having been removed from the data itself:

...

SHOW MORE

Has Missing Values:

Yes

1079304,2,1,1,1,2,1,2,1,1,2
DOI 1080185,10,10,10,8,6,1,8,9,1,4
10.244 1081791,6,2,1,1,1,1,7,1,1,2
1084584,5,4,4,9,2,10,5,6,1,4
1091262,2,5,3,3,6,7,7,5,1,4
1096800,6,6,6,9,6,7,8,1,2
1099510,10,4,3,1,3,3,6,5,2,4
1100524,6,10,10,2,8,10,7,3,3,4
1102573,5,6,5,6,10,1,3,1,1,4
1103608,10,10,10,4,8,1,8,10,1,4
1103722,1,1,1,1,2,1,2,1,2,2

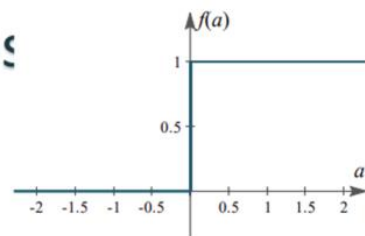
Užduoties punktai:

1. Parsisiųsti ir paruošti duomenis. Klasių žymes (*label*) pakeiskite 0 arba 1. Ištrinkite nereikalingus atributus (stulpelius), pvz., ID. Krūties vėžio duomenyse yra trūkstančių reikšmių, tose vietose yra klaustukai (?), todėl tokius įrašus (eilutes) reikia ištrinti. Duomenų eilutes reikia permaišyti atsitiktine tvarka, kad klasių įrašai būtų atsitiktinai išdėstyti, t. y., nebūtų taip, kad pradžioje tik vienos klasės duomenys, paskui – kitos.
2. Sukurti programą (parašyti kodą), kuri įgyvendintų vieno sigmoidinio neurono mokymo, validavimo ir testavimo procesą, sprendžiant klasifikavimo uždavinį. Reikia įgyvendinti du atvejus: (1) kai neuronas mokomas naudojant paketinį gradientinį nusileidimą, (2) kai neuronas mokomas naudojant stochastinį gradientinį nusileidimą. Turi būti įgyvendintas neurono mokymas, validavimas ir testavimas, todėl turimi duomenys turi būti padalinami į tris aibes. Galima taikyti santykį 80:10:10.

Aktyvacijos funkcijos

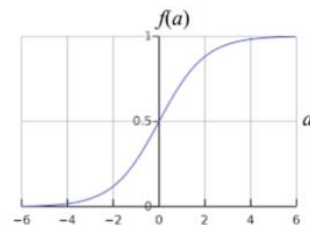
- Slenkstinė

$$f(a) = \begin{cases} 1, & \text{if } a \geq 0 \\ 0, & \text{if } a < 0 \end{cases}$$

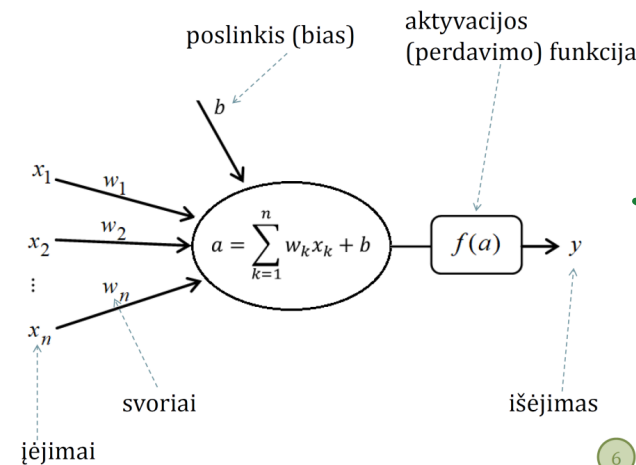


- Sigmoidinė

$$f(a) = \frac{1}{1 + e^{-a}}$$



x_1
 x_2
 x_3
 x_4





<https://eead-csic-compbio.github.io/bioinformatics/session4.html>

Training and Holdout Sets

In practice, data analysts work with three distinct sets of examples: training, validation, and test set



The learning algorithm, trained on the **Training Set**, is not allowed to use examples from either the **Validation Set** or the **Test Set**

We don't want the model to only be accurate at predicting labels of examples the learning algorithm has already seen



Used to train
the model



Used to choose the learning
algorithm and to find the best
hyperparameter configuration

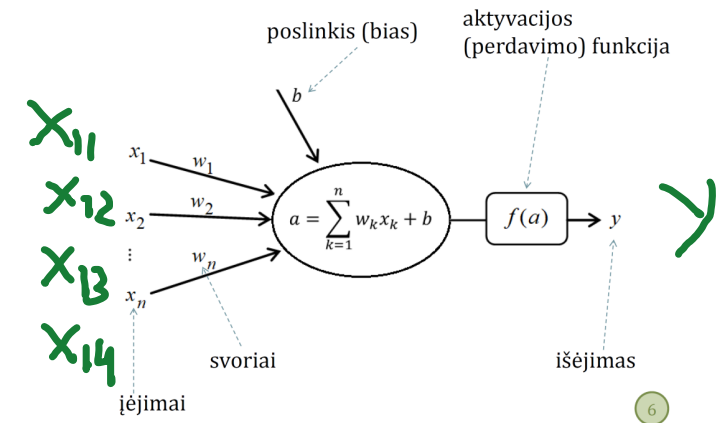


Used to assess
the model before
delivering it

<https://illustrated-machine-learning.github.io/#/machine-learning-engineering/introduction#training-vs-scoring>

Neurono mokymas

- Galimybė **mokytis** yra **esminė intelekto savybė**.
- Tinkamų svorių radimas vadinamas neurono (perceptrono) **mokymu**.
- Duomenų aibė, kuri bus naudojama neuronui mokytis, vadinama **mokymo aibe**.
- Tegul turime m mokymo aibės vektorių $X_i = (x_{i1}, x_{i2}, \dots, x_{in})$, $i = 1, \dots, m$, kuriuos vadinsime **įėjimų vektoriais**.
- Šie vektoriai yra susieti su **norima reikšme** t_i (*target*). Tai norima reakcija į vektorių X_i .
- Sprendžiant klasifikavimo uždavinį, **norimos reikšmės yra klasių numeriai**.



~~150~~
100

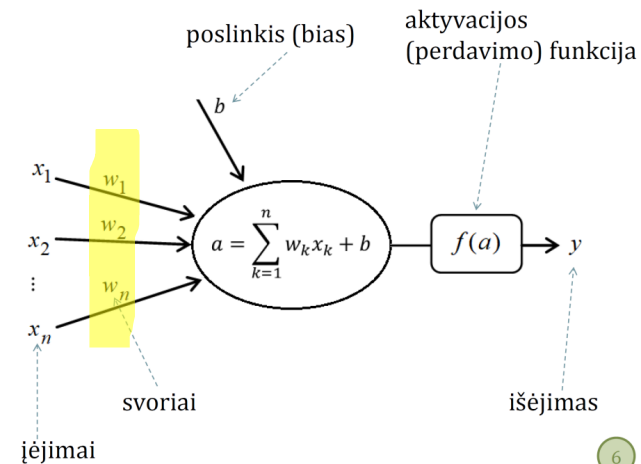
x_{i1}	x_{i2}	x_{i3}	x_{i4}	KLASE
4.6	3.4	1.4	0.3	Iris-setosa
5.0	3.4	1.5	0.2	Iris-setosa
4.4	2.9	1.4	0.2	Iris-setosa
4.9	3.1	1.5	0.1	Iris-setosa
5.4	3.7	1.5	0.2	Iris-setosa
4.8	3.4	1.6	0.2	Iris-setosa
4.8	3.0	1.4	0.1	Iris-setosa
4.3	3.0	1.1	0.1	Iris-setosa
5.8	4.0	1.2	0.2	Iris-setosa
-	-	-	-	-

Neurono mokymas

- Pradžioje svoriams (ir poslinkiui) $W = (w_0, w_1, w_2, \dots, w_n)$ nustatomos **atsitiktinės reikšmės**, čia $w_0 = b$.
- Mokymo procese** svoriai $W = (w_0, w_1, w_2, \dots, w_n)$ keičiami taip, kad tinklo išėjimo reikšmė y_i , gauta į įėjimą pateikus vektorių X_i , būtų kiek galima **artimesnė norimai reikšmei** t_i , t. y., yra neurono veikimo **paklaida** būtų kiek galima **mažesnė**.
- Ši **paklaida** $E(W)$ gali būti apibrėžiama, kaip skirtumų tarp neurono išėjime gautų reikšmių ir norimų reikšmių kvadratų sumos funkcija (čia m mokymo duomenų skaičius):

$$E(W) = \sum_{i=1}^m (t_i - y_i)^2$$

- Reikalinga **neurono mokymosi taisyklė** (*learning rule*), t. y. pagal kokią formulę bus keičiami (atnaujinami svoriai ir poslinkis).

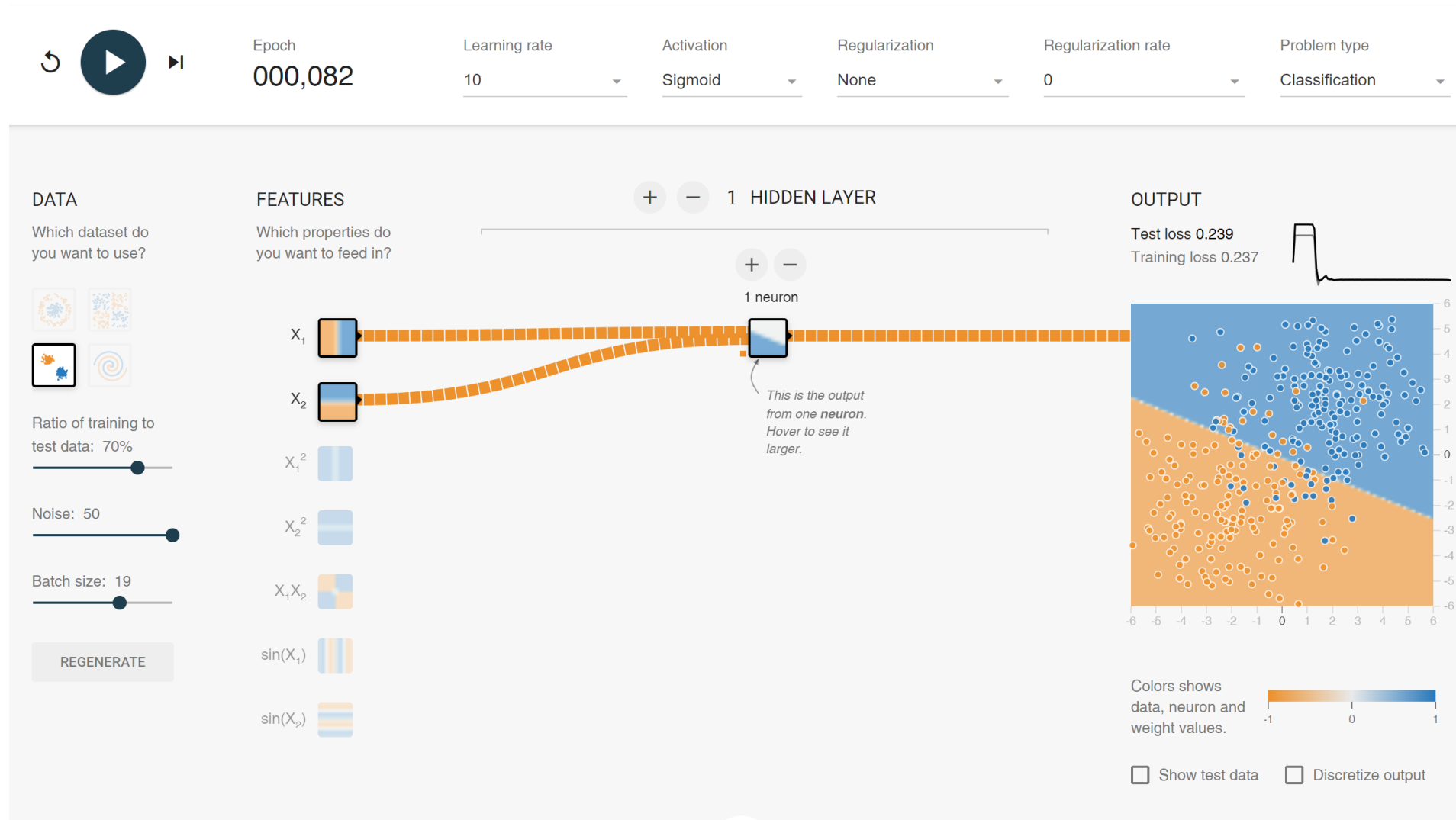


	x_{11}	x_{12}	x_{13}	x_{14}	KLASE
→	4.6	3.4	1.4	0.3	Iris-setosa
	5.0	3.4	1.5	0.2	Iris-setosa
	4.4	2.9	1.4	0.2	Iris-setosa
	4.9	3.1	1.5	0.1	Iris-setosa
100 {	5.4	3.7	1.5	0.2	Iris-setosa
	4.8	3.4	1.6	0.2	Iris-setosa
	4.8	3.0	1.4	0.1	Iris-setosa
	4.3	3.0	1.1	0.1	Iris-setosa
	5.8	4.0	1.2	0.2	Iris-setosa
	-	-	-	-	-

Neurono mokymas

- Taigi, neurono mokymo eigoje reikia **minimizuoti paklaidos funkciją** $E(W)$, kuri dar vadinama **nuostolių** funkcija (*loss function, cost function*).
- Jeigu ši funkcija yra diferencijuojama pagal svorius, jos minimumą galima rasti **gradientiniais optimizavimo metodais**.
- Patogumo dėlei, dažnai minimizuojama **tokios išraiškos funkcija**:

$$E(W) = \frac{1}{2} \sum_{i=1}^m (t_i - y_i)^2$$



<https://playground.tensorflow.org/>
<http://neuralnetworksanddeeplearning.com/index.html>

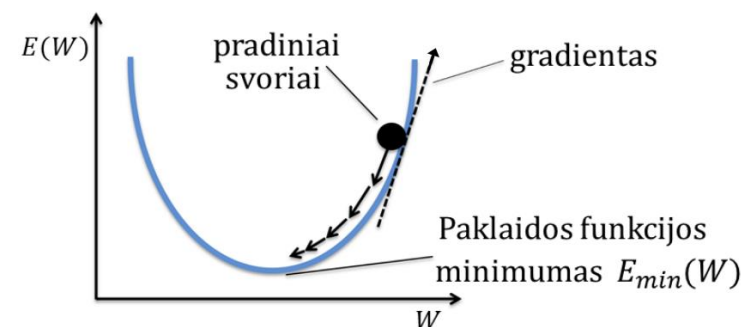
Neurono mokymas gradientinio nusileidimo būdu

- Iš pradžių **generuojamos atsitiktinės svorių** w_k reikšmės, pvz., intervale $(0, 1)$ arba $(-1, 1)$. Tačiau gali būti taikomos ir kitos pradinių svorių parinkimo strategijos.
- Tada gradientinio nusileidimo algoritmu judama **antigradiento** kryptimi, **svorių reikšmės keičiant** pagal iteracinę formulę

$$w_k := w_k - \eta \frac{\partial E(W)}{\partial w_k}, k = 0, \dots, n$$

η – yra teigiamas daugiklis, kuris vadinamas **mokymo greičiu** (*learning rate*) ir kuriuo reguliuojamas gradientinio optimizavimo žingsnio ilgis.

Gradientinis nusileidimas



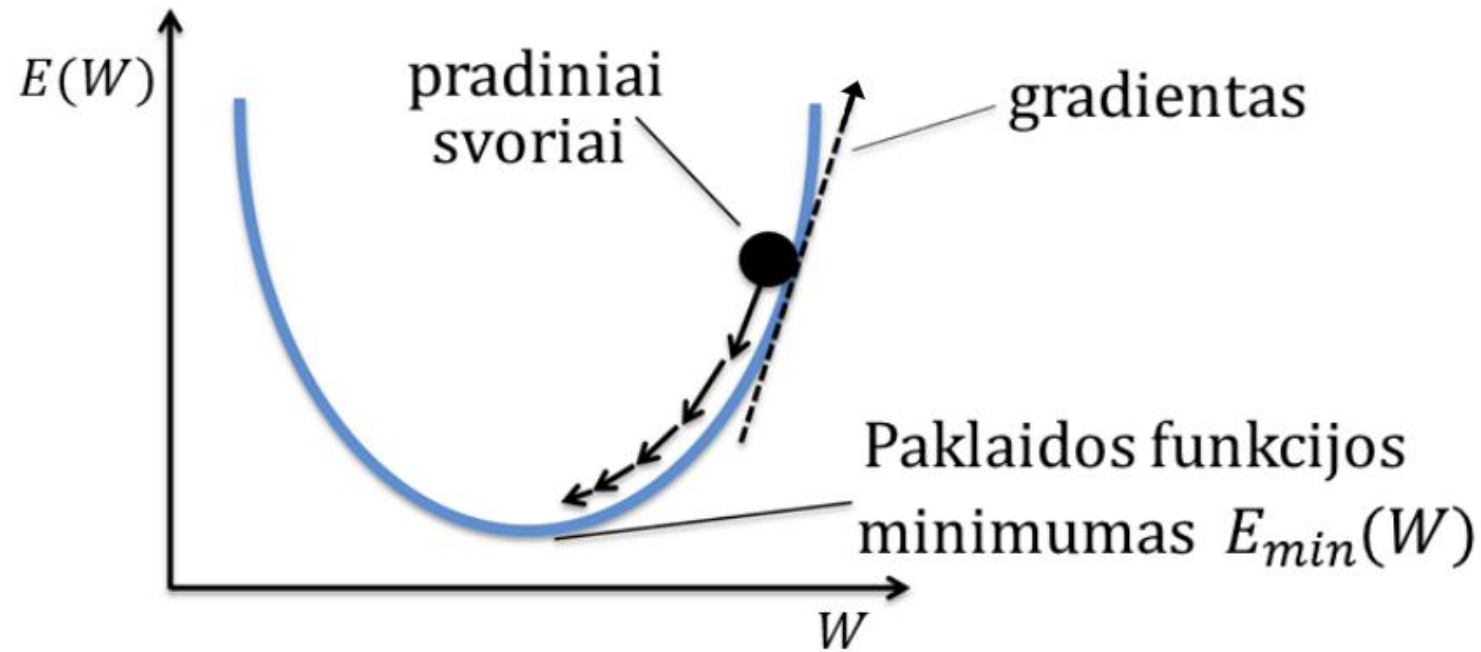
Judama antigradiento kryptimi.

24

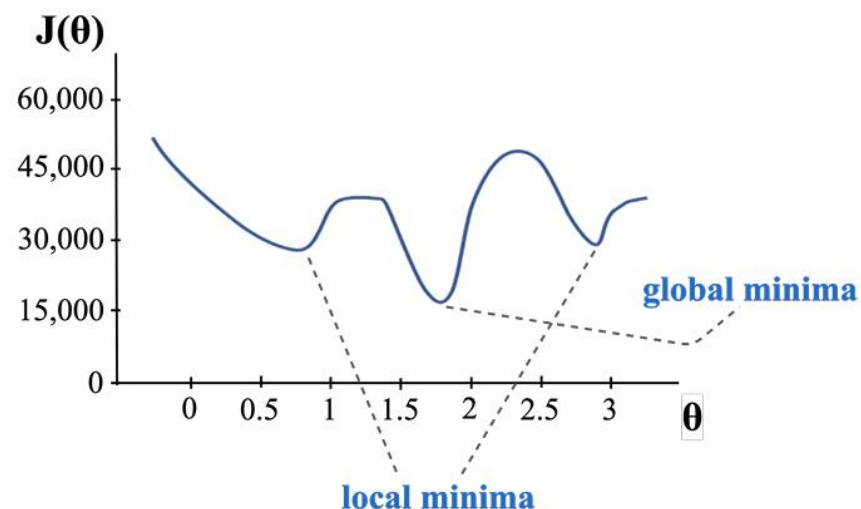
100 { 4.6, 3.4, 1.4, 0.3, Iris-setosa
5.0, 3.4, 1.5, 0.2, Iris-setosa
4.4, 2.9, 1.4, 0.2, Iris-setosa
4.9, 3.1, 1.5, 0.1, Iris-setosa
5.4, 3.7, 1.5, 0.2, Iris-setosa
4.8, 3.4, 1.6, 0.2, Iris-setosa
4.8, 3.0, 1.4, 0.1, Iris-setosa
4.3, 3.0, 1.1, 0.1, Iris-setosa
5.8, 4.0, 1.2, 0.2, Iris-setosa
- - - - -

23

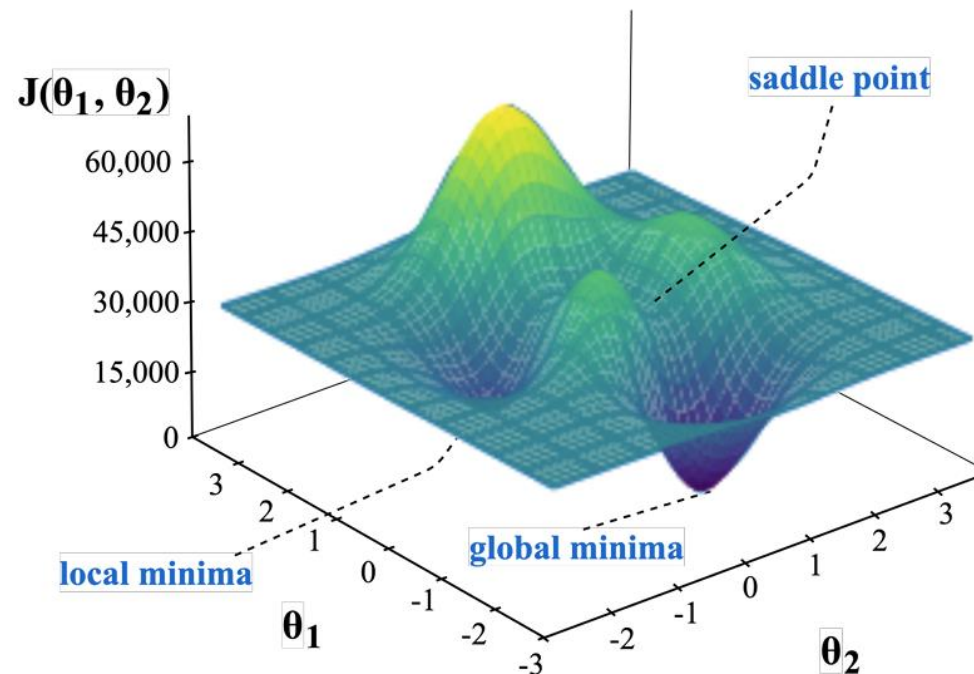
Gradientinis nusileidimas



Judama antigradiento kryptimi.



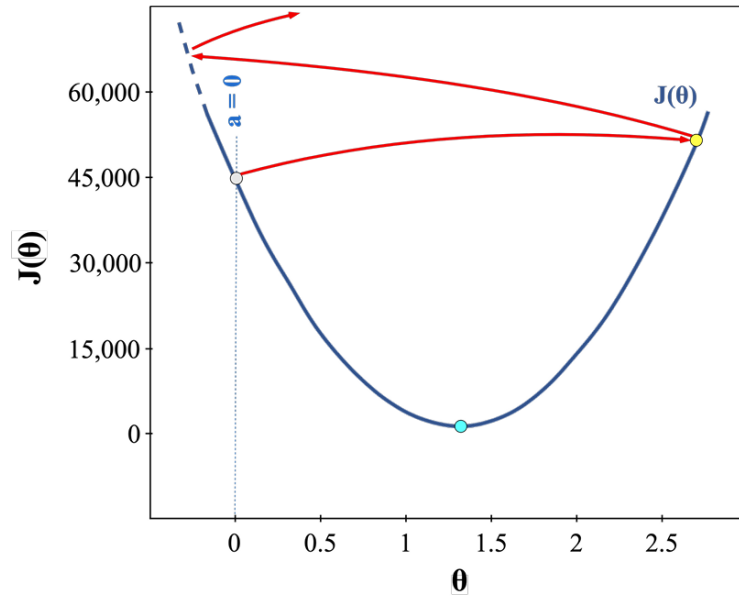
(c) Non-convex, one parameter; note that there is no saddle point in function of 1 parameter (saddle point occurs when gradient is zero, but point is minimum in one direction and maximum in another).



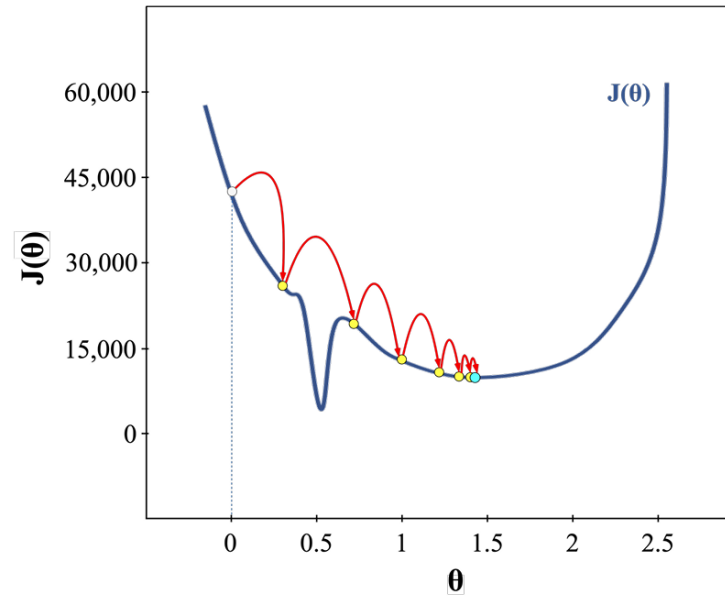
(d) Non-convex, two parameters

<https://themlsbook.com/read>

Andrew Wolf. **Machine Learning Simplified: A gentle introduction to supervised learning**



(a) Gradient descent missing global minimum on a convex cost function due to a very large learning rate.



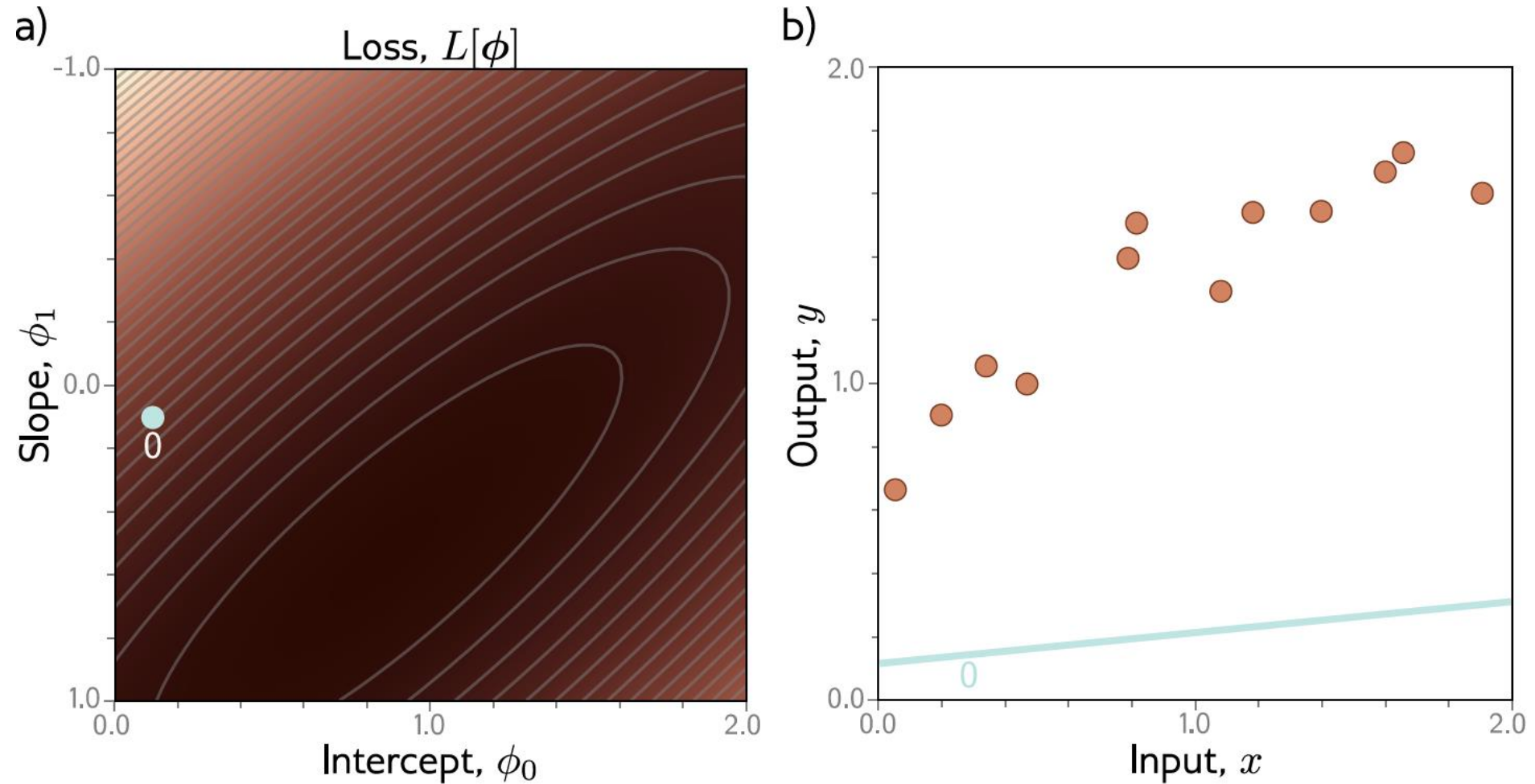
(b) Gradient Descent missing global minimum on a non-convex cost function due to a very large learning rate.

Figure 3.15: *Comparison of the gradient descent algorithm missing a global minimum on a convex cost function (left) and a non-convex cost function (right). If the learning rate l (and the step size with it) is too large, for a convex cost function we can (plausibly) "jump over" the minima we are trying to reach, while for a non-convex cost function we can skip a global optimum.*

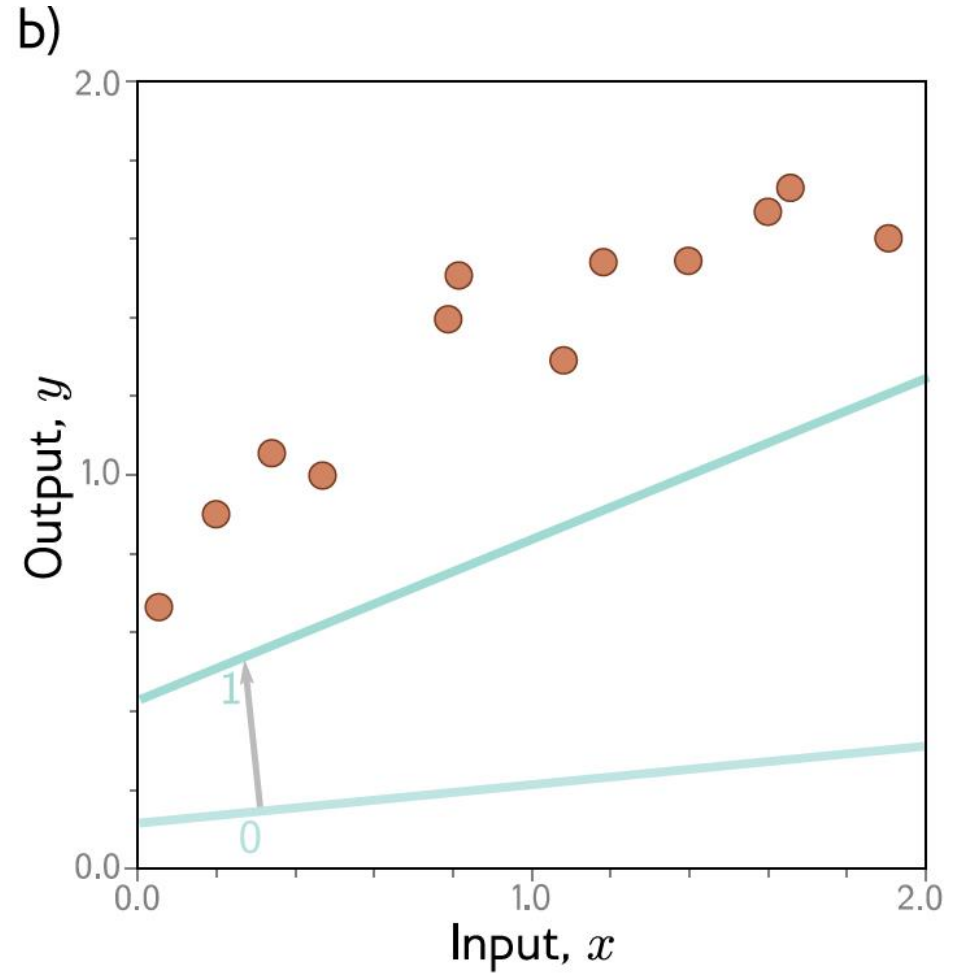
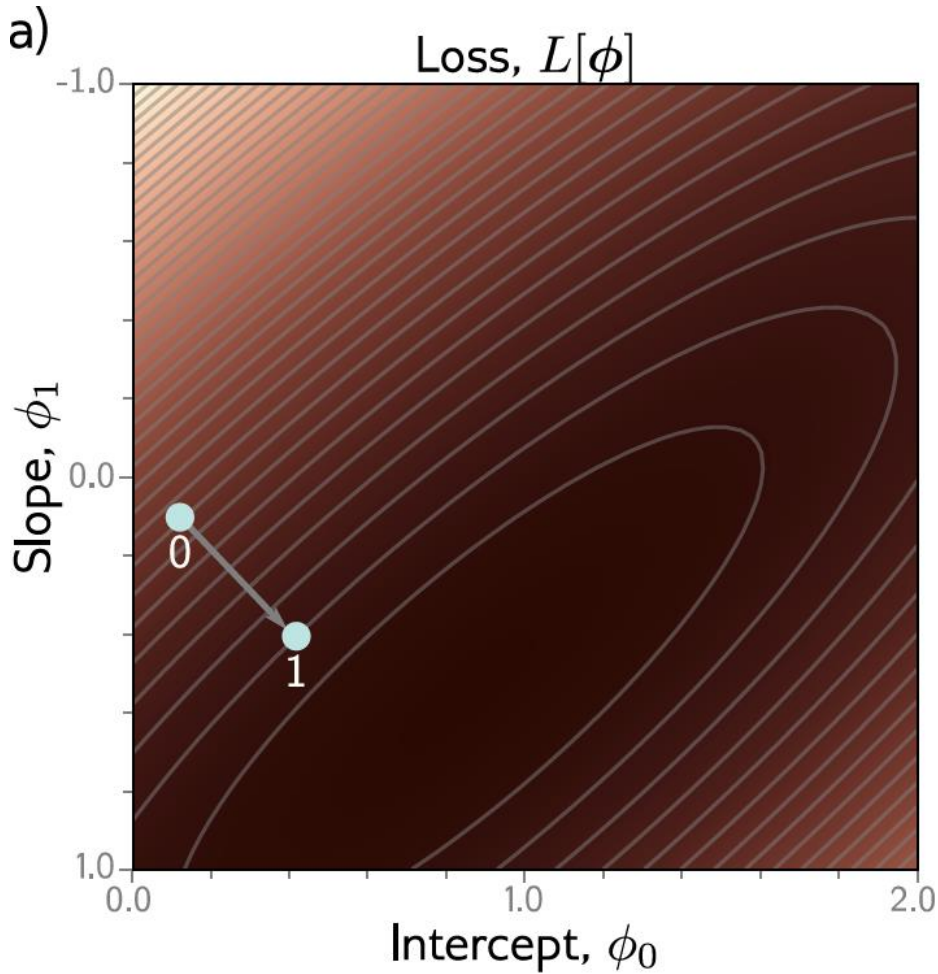
<https://themlsbook.com/read>

Andrew Wolf. **Machine Learning Simplified: A gentle introduction to supervised learning**

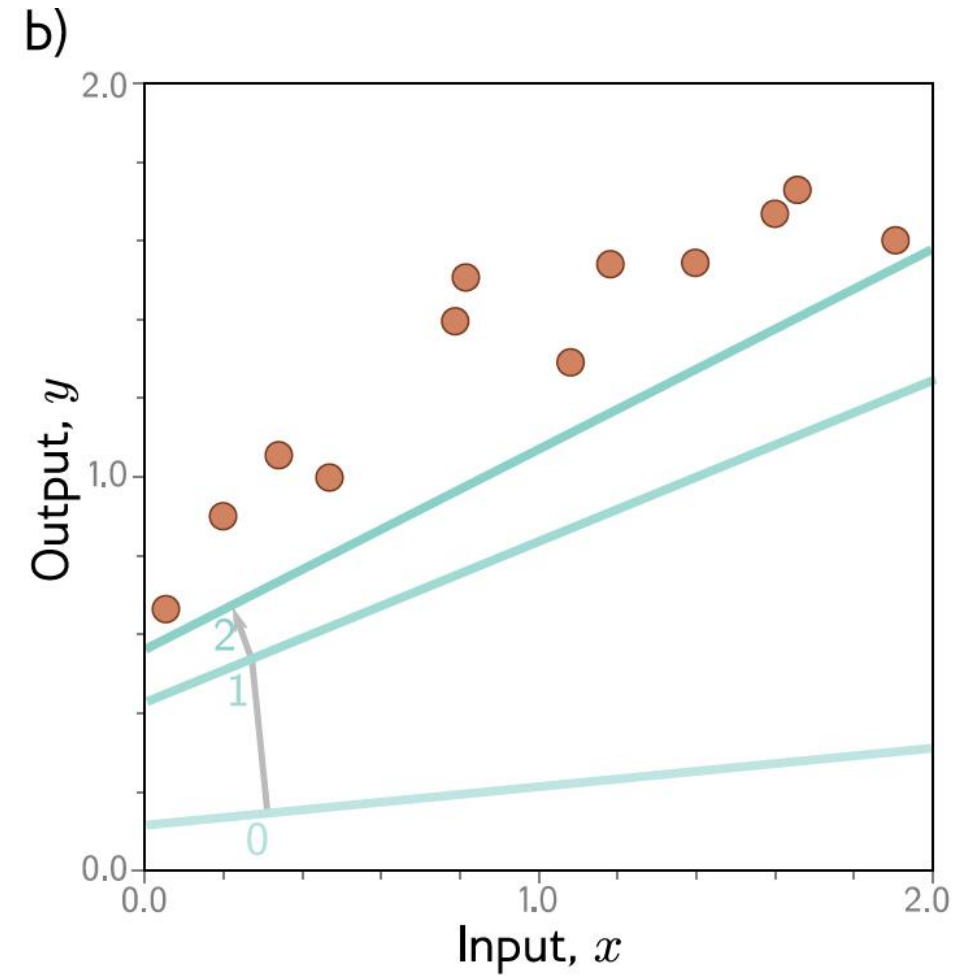
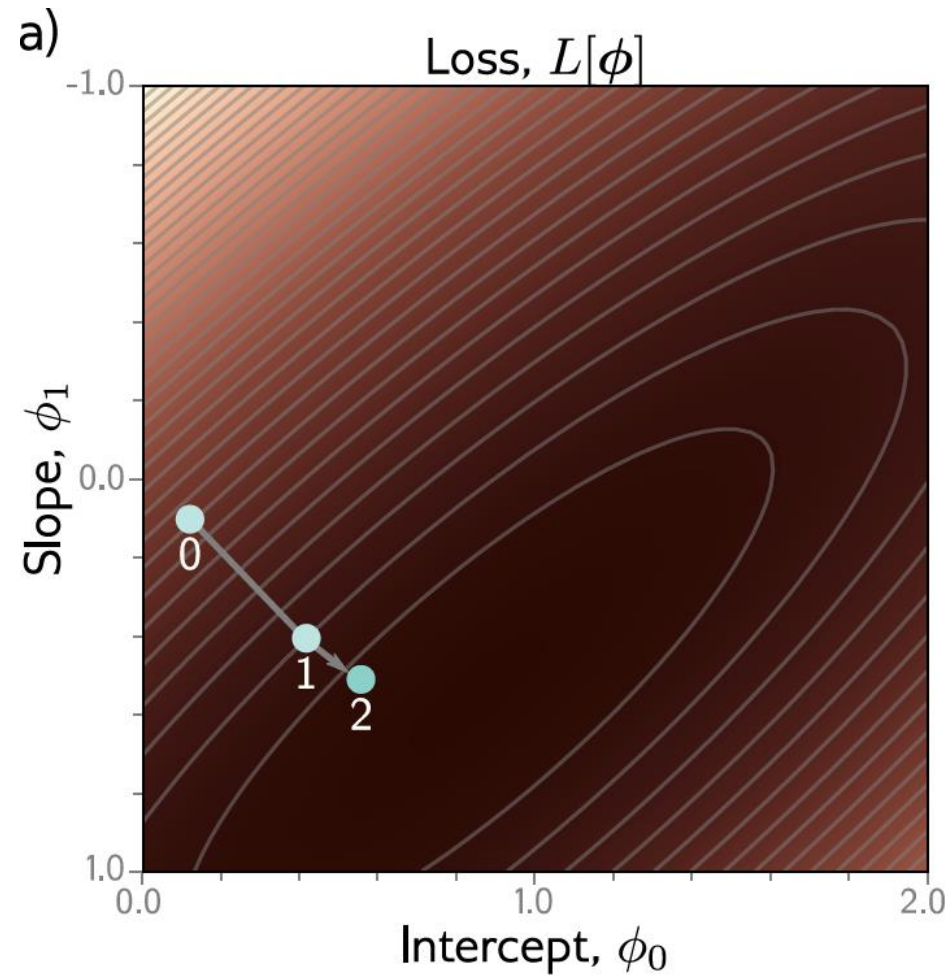
Example: 1D Linear regression training



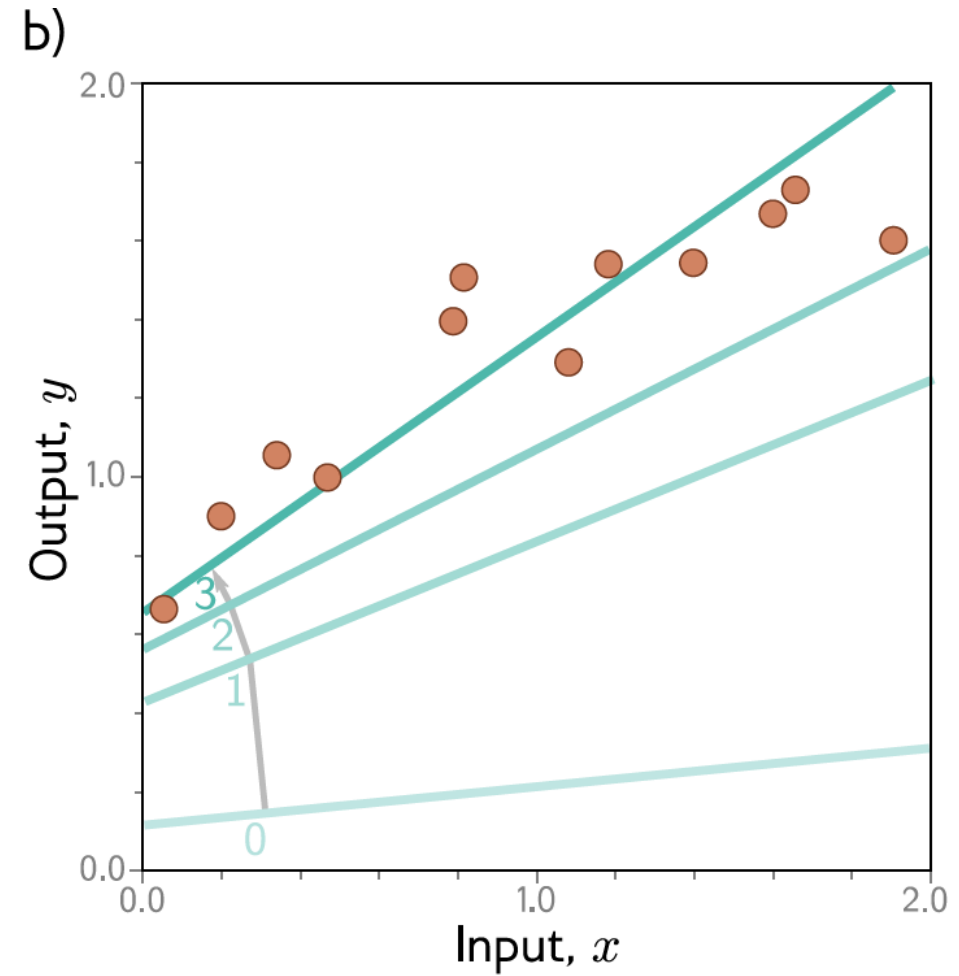
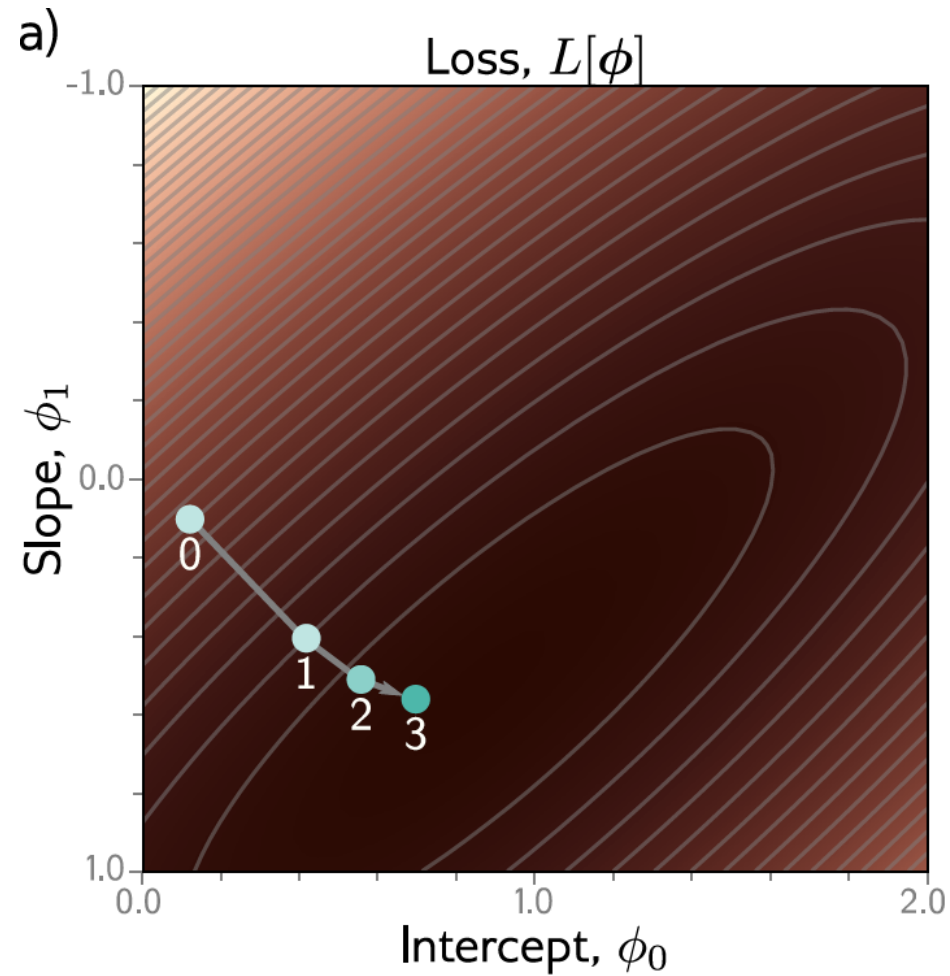
Example: 1D Linear regression training



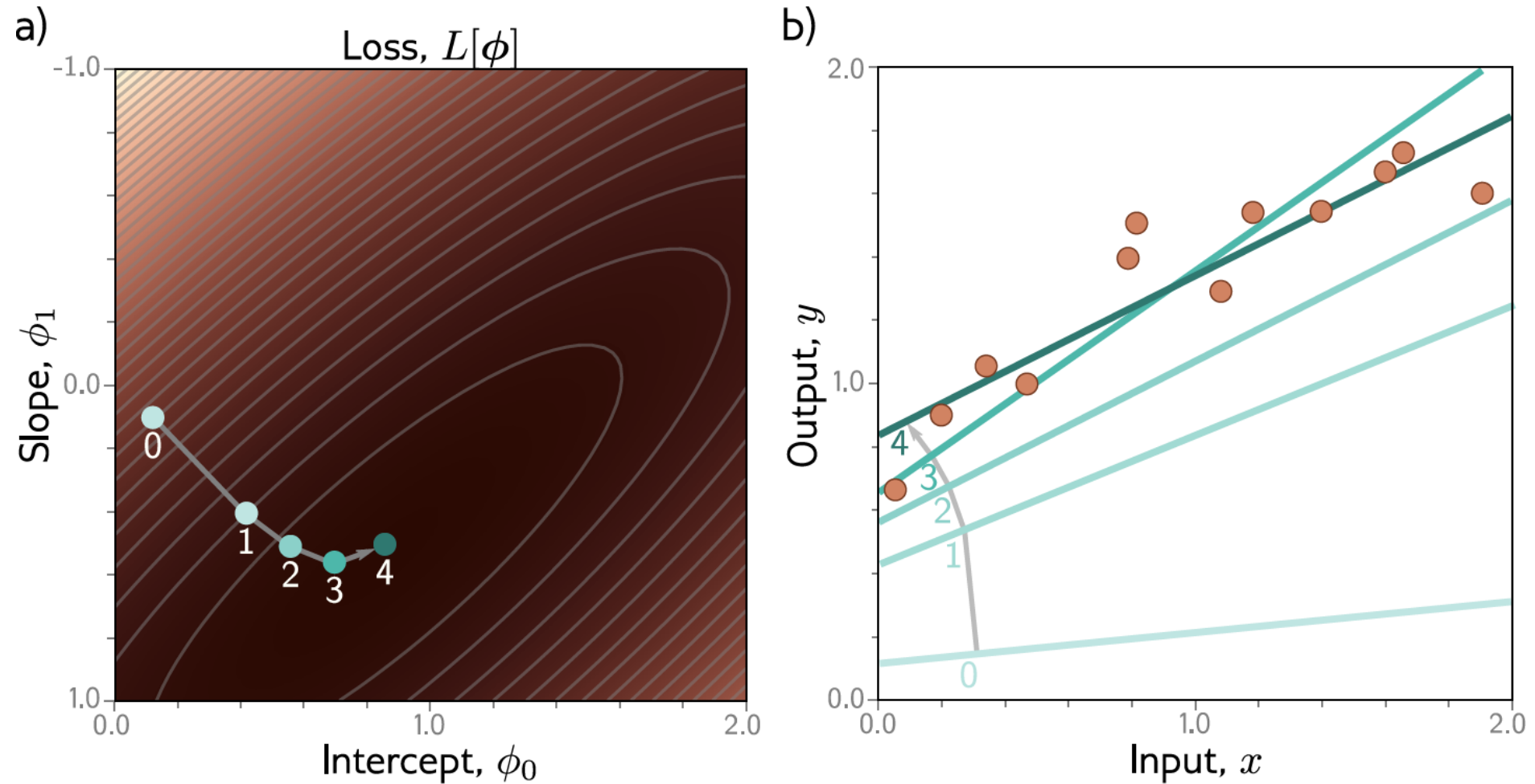
Example: 1D Linear regression training

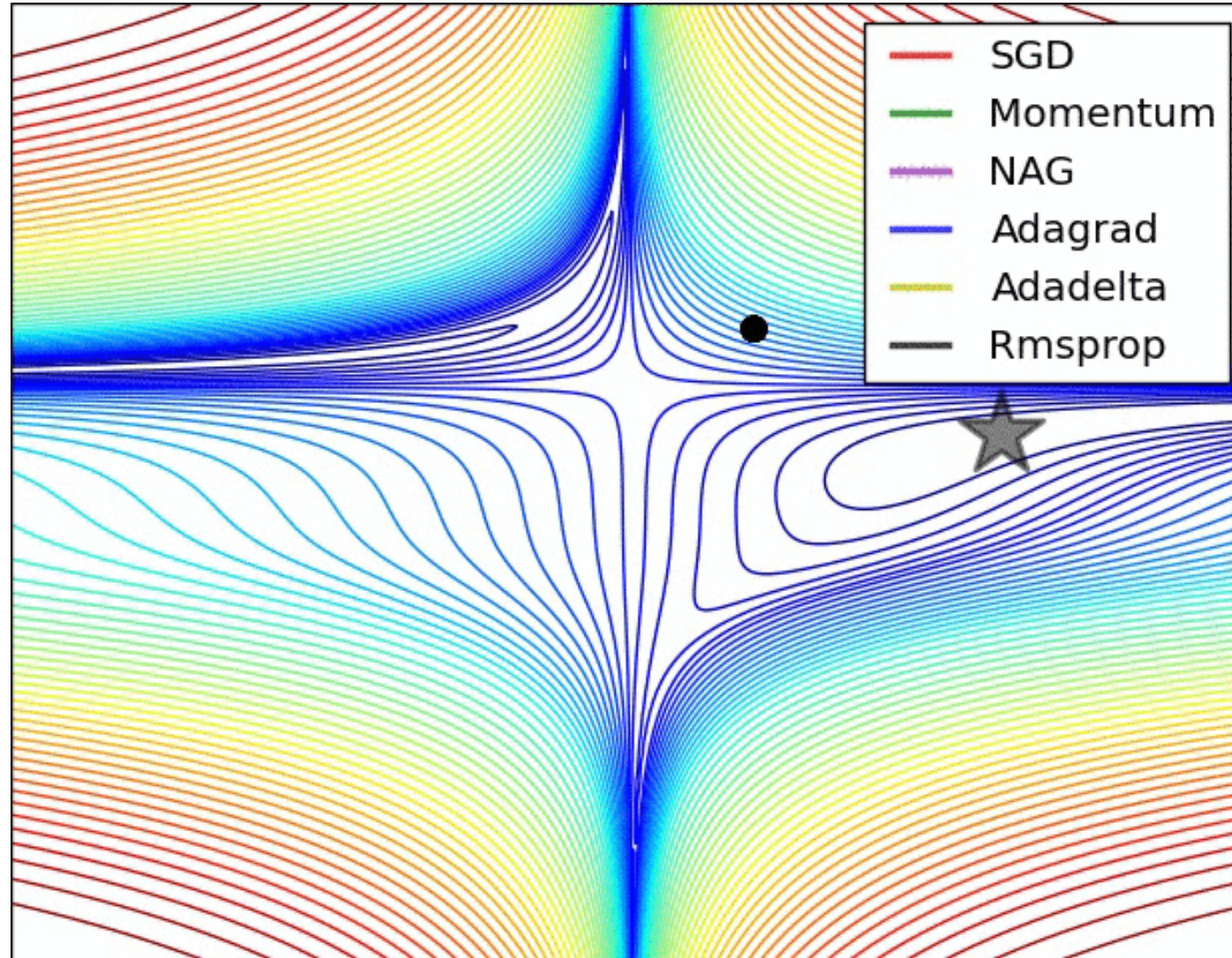


Example: 1D Linear regression training



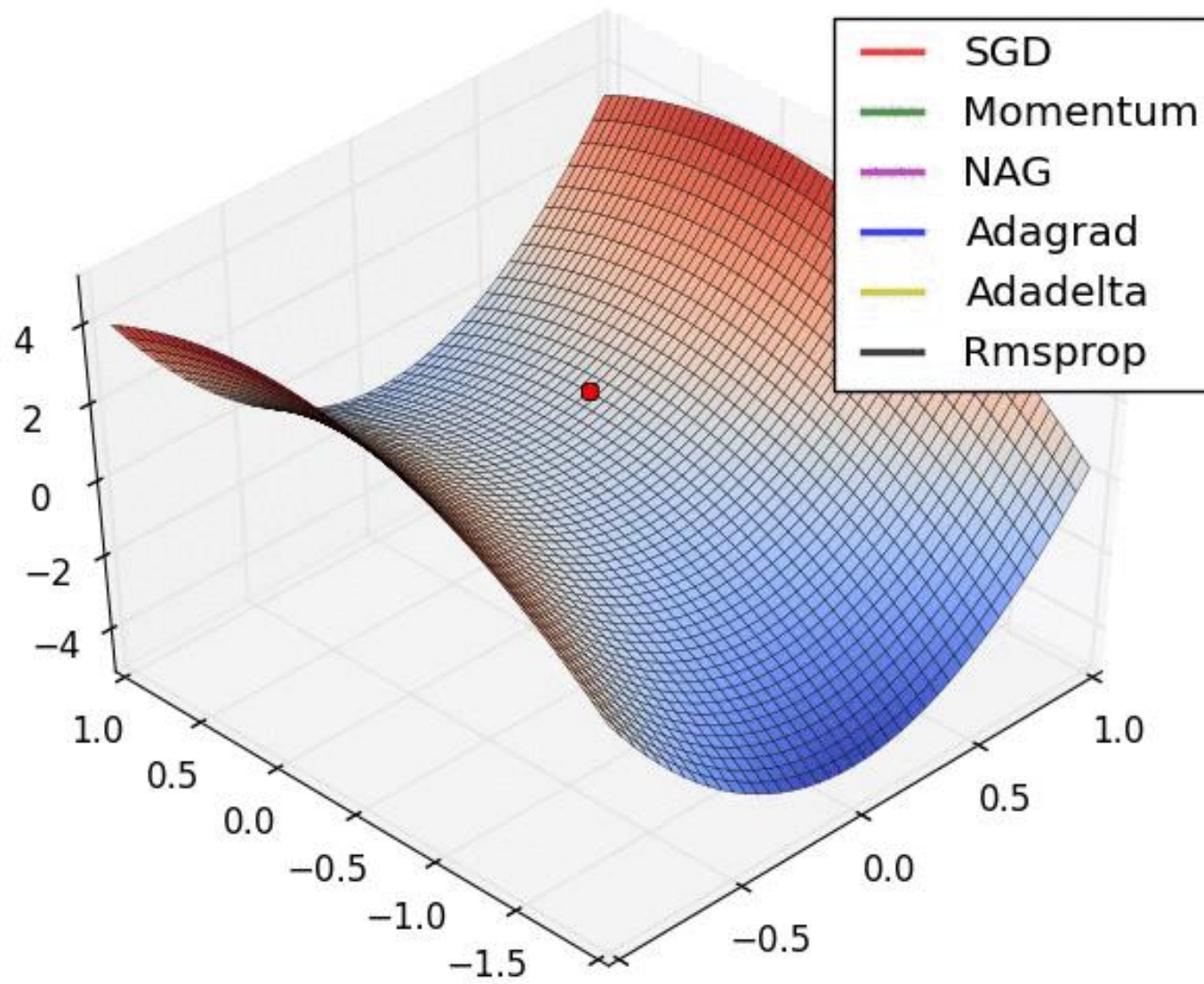
Example: 1D Linear regression training





<https://cs231n.github.io/neural-networks-3/>

Images credit: [Alec Radford](#)



<https://cs231n.github.io/neural-networks-3/>

Images credit: [Alec Radford](#)

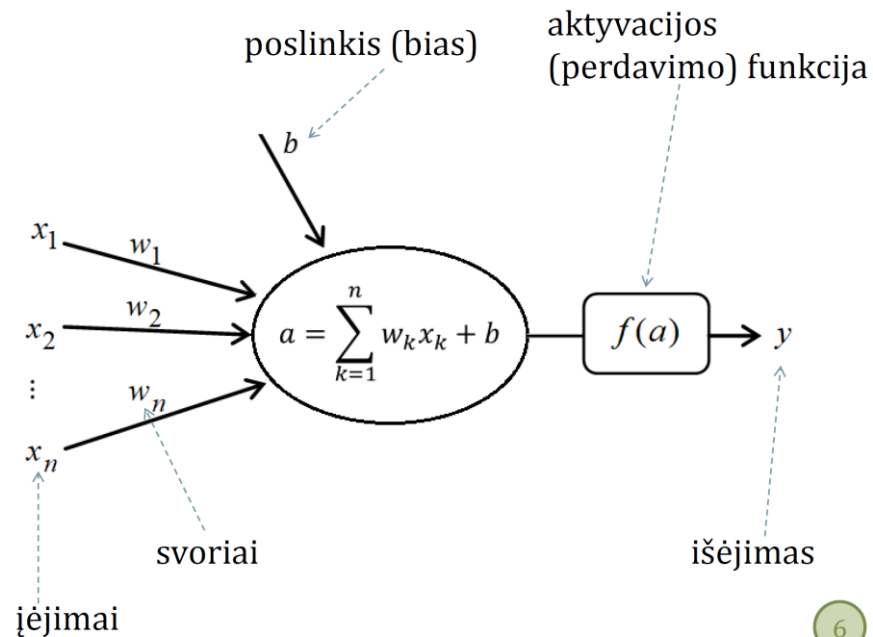
A. $W = a * \text{random}(-y, y)$

B. $W = a * \text{random}(\text{in_num}, y_num)$

C. $W = a * \text{random}(\text{in_num}, y_num); a = 1 / \sqrt{\text{in_num}}$ – *Xavier initialization (*)*

D. $W = a * \text{random}(\text{in_num}, y_num); a = 1 / \sqrt{\text{in_num} / 2}$ – *He initialization (**)*

...



* Understanding the difficulty of training deep feedforward neural networks,

<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.207.2059&rep=rep1&type=pdf>

** Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification

<https://arxiv.org/abs/1502.01852>

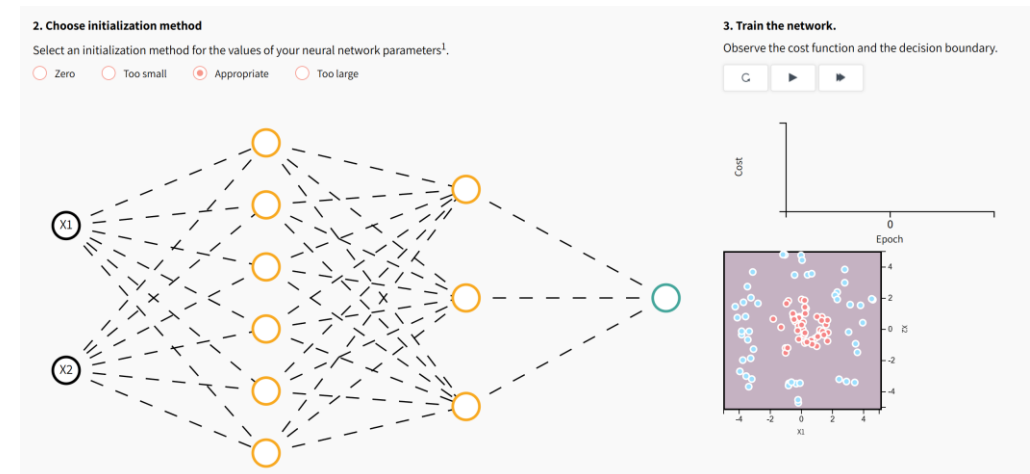
I The importance of effective initialization

To build a machine learning algorithm, usually you'd define an architecture (e.g. Logistic regression, Support Vector Machine, Neural Network) and train it to learn parameters. Here is a common training process for neural networks:

1. Initialize the parameters
2. Choose an *optimization algorithm*
3. Repeat these steps:
 1. Forward propagate an input
 2. Compute the cost function
 3. Compute the gradients of the cost with respect to parameters using backpropagation
 4. Update each parameter using the gradients, according to the optimization algorithm

Then, given a new data point, you can use the model to predict its class.

The initialization step can be critical to the model's ultimate performance, and it requires the right method. To illustrate this, consider the three-layer neural network below. You can try initializing this network with different methods and observe the impact on the learning.



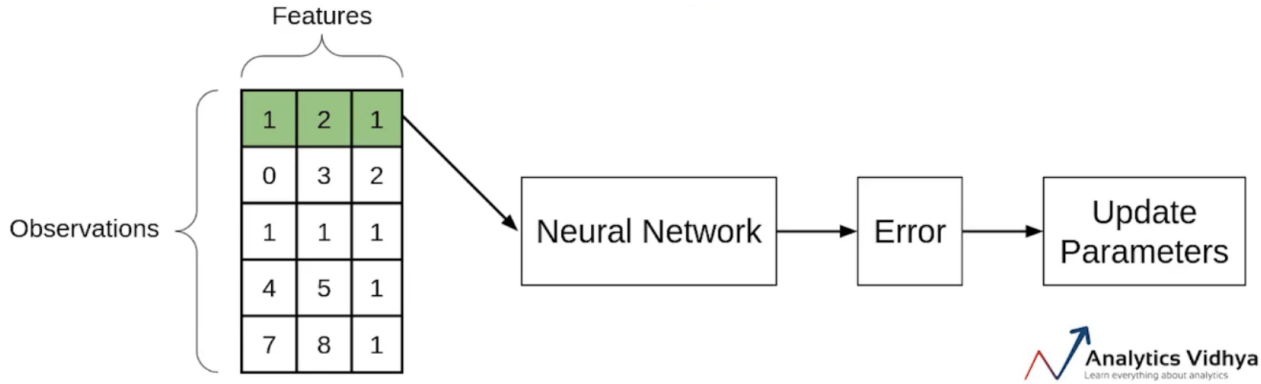
<https://www.deeplearning.ai/ai-notes/initialization/index.html>

Gradientinio nusileidimo tipai

- ✓ • **Paketinis gradientinis nusileidimas** (*batch gradient descent*),
- ✓ • **Stochastinis gradientinis nusileidimas** (*stochastic gradient descent*),
- **Mažų paketų gradientinis nusileidimas** (*mini-batch gradient descent*).

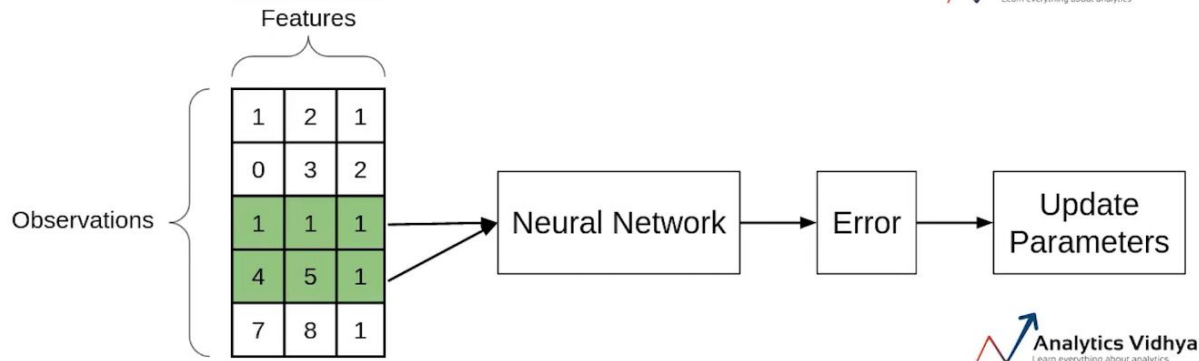
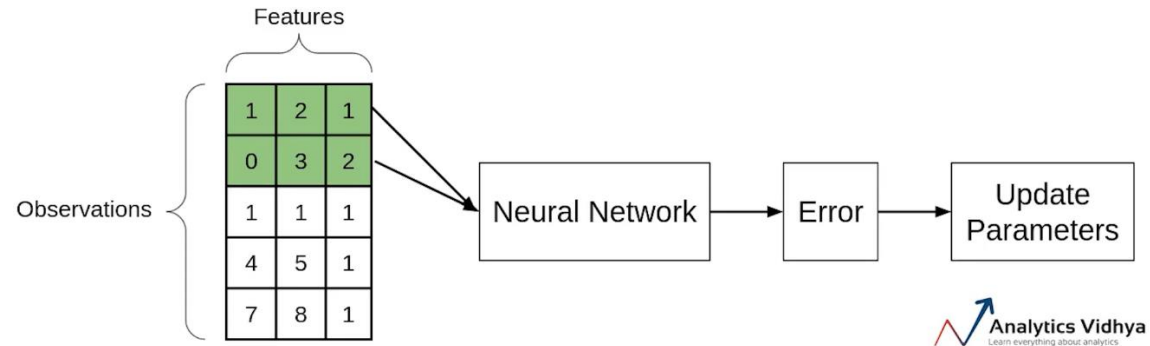
Gradientinio nusileidimo **tipas priklauso nuo to**, į **kokią kiekį mokymo duomenų** atsižvelgiama vienai mokymo algoritmo **iteracijai** atlikti.

Stochastic Gradient Descent (SGD)

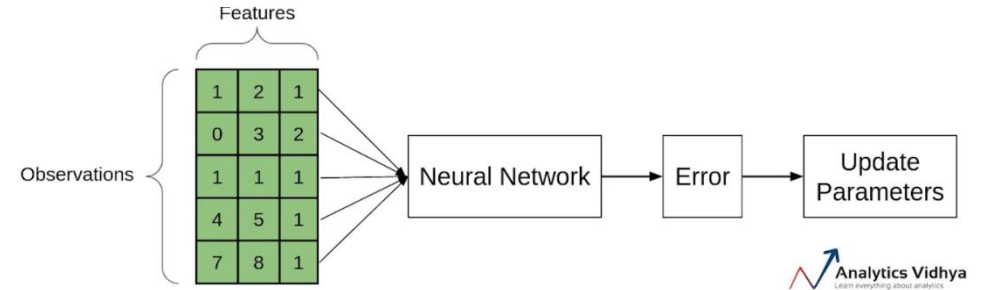


$$w_k := w_k - \eta \frac{\partial E(W)}{\partial w_k}, k = 0, \dots, n$$

Mini-batch Gradient Descent



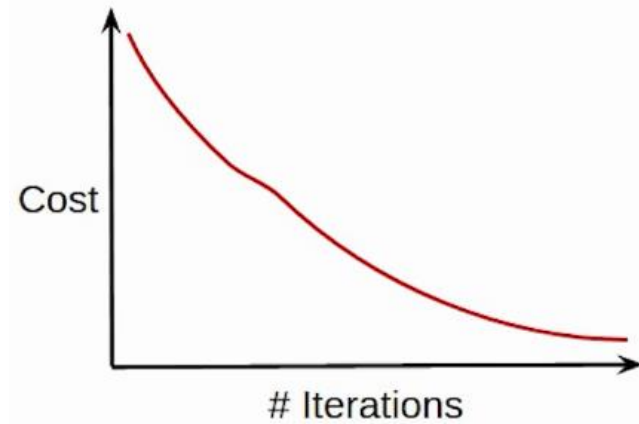
Batch Stochastic Gradient Descent



Batch Gradient Descent

- Entire dataset for updation

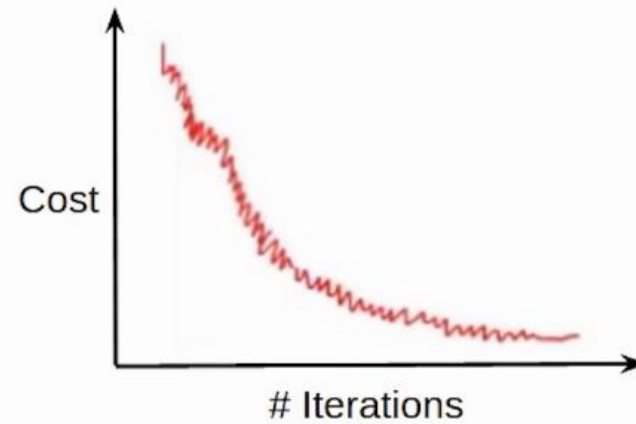
- Cost function reduces smoothly



Stochastic Gradient Descent (SGD)

- Single observation for updation

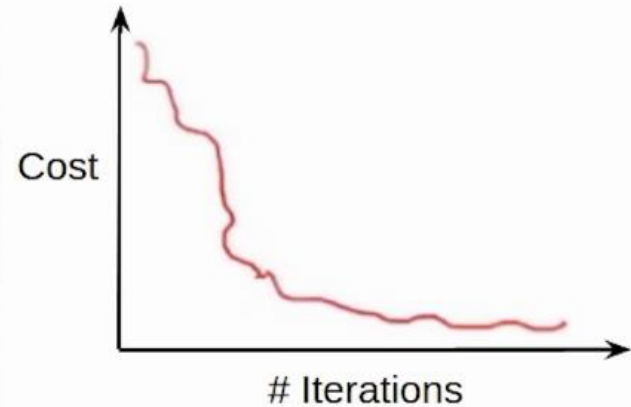
- Lot of variations in cost function



Mini-Batch Gradient Descent

- Subset of data for updation

- Smoother cost function as compared to SGD



Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
• Entire dataset for updation	• Single observation for updation	• Subset of data for updation
Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
• Entire dataset for updation • Cost function reduces smoothly • Computation cost is very high	• Single observation for updation • Lot of variations in cost function • Computation time is more	• Subset of data for updation • Smoother cost function as compared to SGD • Computation time is lesser than SGD • Computation cost is lesser than Batch Gradient Descent

Batch Gradient Descent

Iterative optimization algorithm for finding a local minimum of any differentiable function
It relies on the properties of the gradients

The derivative $f'(x)$ of a function $f(x)$ represents the slope of $f(\cdot)$ in x .

It allows to determine how to modify x in order to make a small improvement on our final loss function.

$$f(x + \epsilon) \sim f(x) + \epsilon f'(x)$$

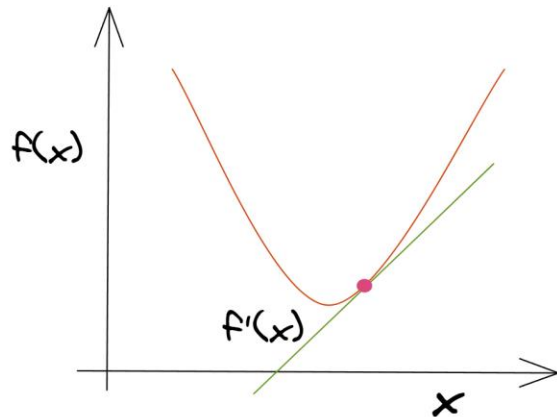
Since we aim to minimize our loss function, we go in the opposite direction of the gradient.

$$f(x - \epsilon \text{sign}(f'(x))) < f(x)$$

Gradient: Vector of partial derivatives

Randomly initialize the weights W

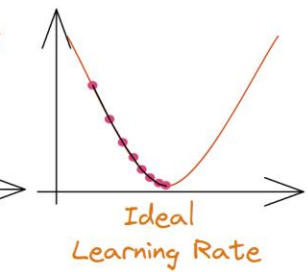
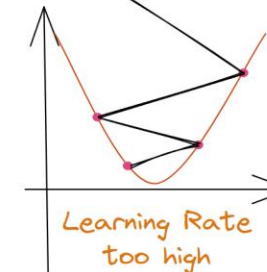
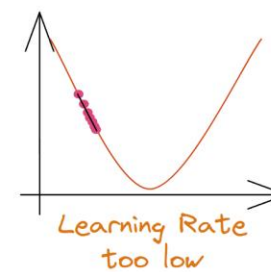
for a fixed number of iterations...
Calculate the gradients G
Update: $W = W - \epsilon G$



$$x' = x - \epsilon \nabla_x f(x)$$

Learning rate!

This represents the step size to take in the opposite direction of the gradient



it is also known as
BATCH GRADIENT DESCENT

Stochastic Gradient Descent

Gradient Descent is too expensive..

SGD approximates the gradient using only one datapoint!

Much faster than GD

With a sufficiently high number of iterations, it is able to reach a satisfactory solution

Randomly initialize the weights W

for a fixed number of iterations...

Draw random example with replacement x
Calculate the gradients G wrt to x
Update: $W = W - \epsilon G$

Minibatch Gradient Descent

It leverages the advantages of the previous methods

It performs an update for every mini-batch of n training examples!

Reduces the variance of the parameter updates

Leverages very efficient matrix computation thanks to pre-implemented libraries

Randomly initialize the weights W

for a fixed number of iterations..
for batch in batches:

Calculate the gradients G wrt to x
Update: $W = W - \epsilon G$

Because this estimate of the gradient is noisy, the weights may not move precisely down the gradient at each iteration. As we shall see, this “noise” at each iteration can be advantageous. Stochastic learning is generally the preferred method for basic backpropagation for the following three reasons:

Advantages of Stochastic Learning

1. Stochastic learning is usually *much* faster than batch learning.
2. Stochastic learning also often results in better solutions.
3. Stochastic learning can be used for tracking changes.

Stochastic learning is most often *much* faster than batch learning particularly on large redundant datasets. The reason for this is simple to show. Consider the simple case where a training set of size 1000 is inadvertently composed of 10 identical copies of a set with 100 samples. Averaging the gradient over all 1000

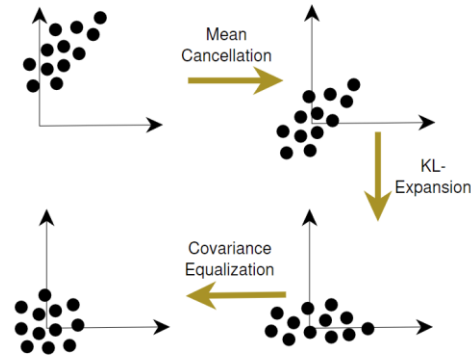


Fig. 3. Transformation of inputs.

4.3 Normalizing the Inputs

Convergence is usually faster if the average of each input variable over the training set is close to zero. To see this, consider the extreme case where all the inputs are positive. Weights to a particular node in the first weight layer are updated by an amount proportional to δx where δ is the (scalar) error at that node and x is the input vector (see equations (5) and (10)). When all of the components of an input vector are positive, all of the updates of weights that feed into a node will be the same sign (i.e. $\text{sign}(\delta)$). As a result, these weights can only all decrease or all increase *together* for a given input pattern. Thus, if a weight vector must change direction it can only do so by zigzagging which is inefficient and thus very slow.

Transforming the Inputs

1. The average of each input variable over the training set should be close to zero.
2. Scale input variables so that their covariances are about the same.
3. Input variables should be uncorrelated if possible.

<https://yann.lecun.com/exdb/publis/pdf/lecun-98b.pdf>

LeCun, Y., Bottou, L., Orr, G. B., & Müller, K. R. (2002). Efficient backprop. In *Neural networks: Tricks of the trade* (pp. 9-50). Berlin, Heidelberg: Springer Berlin Heidelberg.

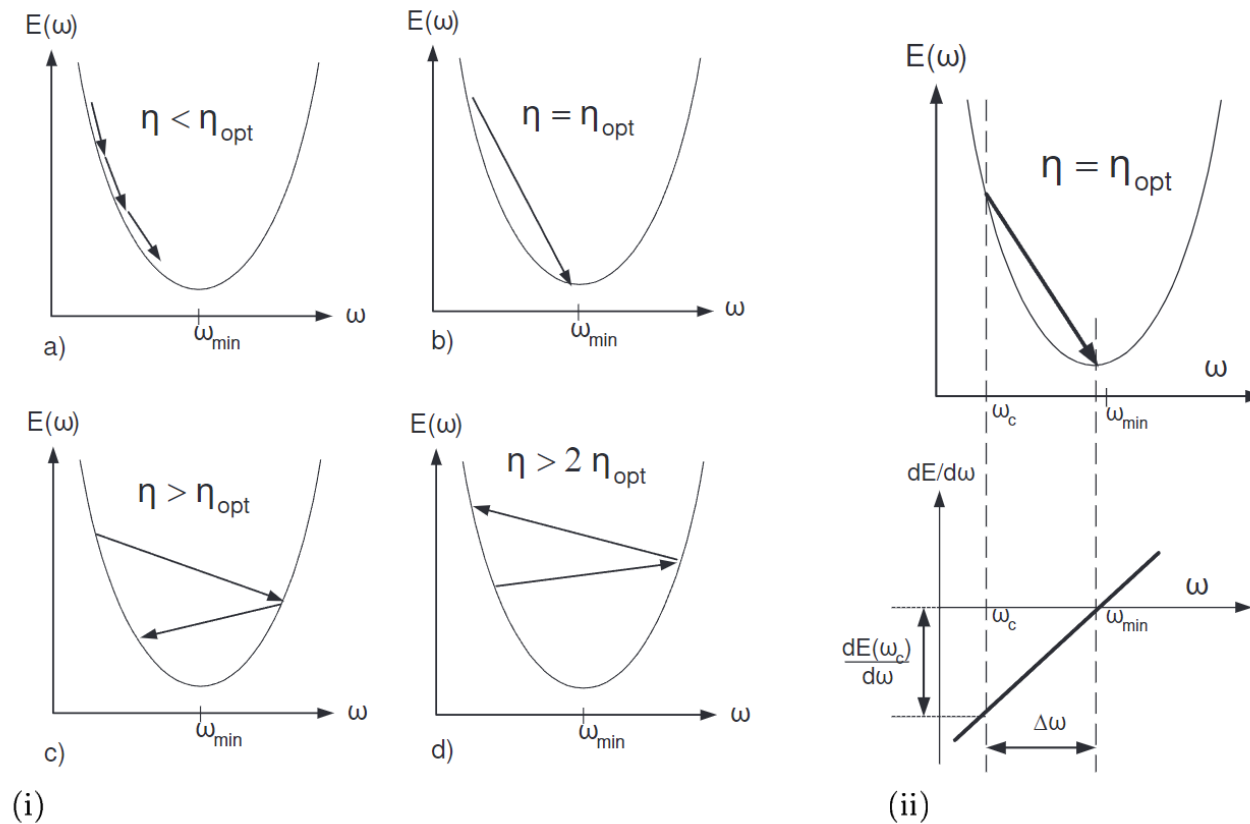
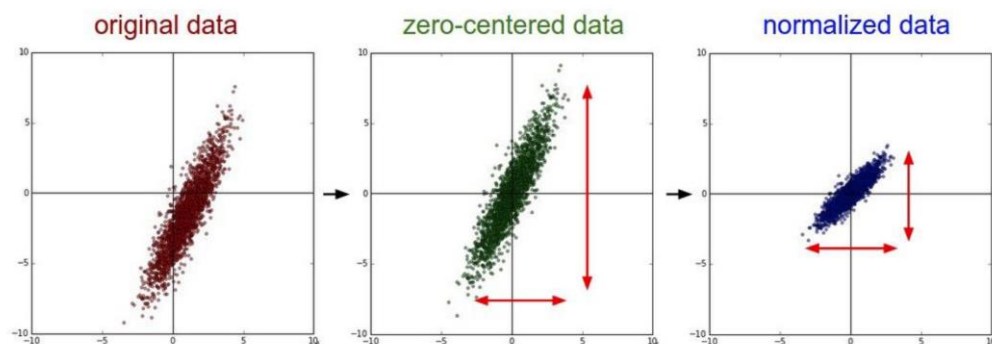


Fig. 6. Gradient descent for different learning rates.

<https://yann.lecun.com/exdb/publis/pdf/lecun-98b.pdf>

LeCun, Y., Bottou, L., Orr, G. B., & Müller, K. R. (2002). Efficient backprop. In *Neural networks: Tricks of the trade* (pp. 9-50). Berlin, Heidelberg: Springer Berlin Heidelberg.

Last time: Data Preprocessing

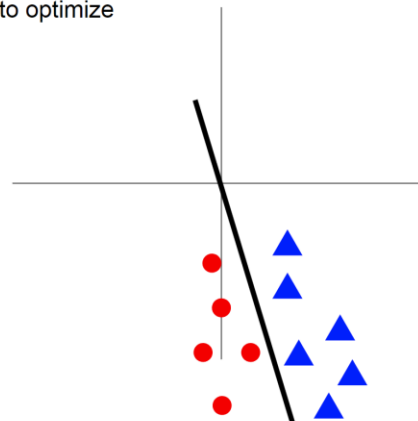


Fei-Fei Li & Justin Johnson & Serena Yeung

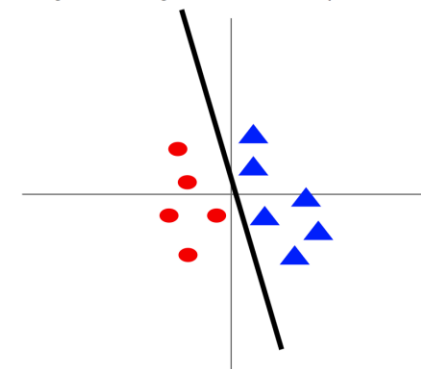
Lecture 7 - 8 April 25, 2017

Last time: Data Preprocessing

Before normalization: classification loss
very sensitive to changes in weight matrix;
hard to optimize



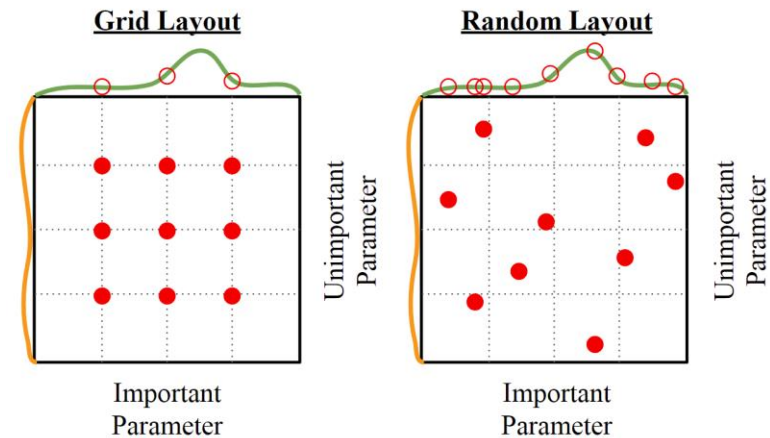
After normalization: less sensitive to small
changes in weights; easier to optimize



Fei-Fei Li & Justin Johnson & Serena Yeung

Lecture 7 - 9 April 25, 2017

Last time: Hyperparameter Search



Coarse to fine search

```
val_acc: 0.412000, lr: 1.405206e-04, reg: 4.793564e-01, (1 / 100)
val_acc: 0.214000, lr: 7.231888e-06, reg: 2.321281e-04, (2 / 100)
val_acc: 0.208000, lr: 2.119571e-06, reg: 8.011857e+01, (3 / 100)
val_acc: 0.196000, lr: 1.551131e-05, reg: 4.374936e-05, (4 / 100)
val_acc: 0.079000, lr: 1.753300e-05, reg: 1.200424e+03, (5 / 100)
val_acc: 0.223000, lr: 4.215128e-05, reg: 4.196174e+01, (6 / 100)
val_acc: 0.241000, lr: 1.750259e-04, reg: 2.110807e-04, (7 / 100)
val_acc: 0.241000, lr: 6.749231e-05, reg: 4.226413e+01, (8 / 100)
val_acc: 0.482000, lr: 4.296863e-04, reg: 6.642555e-01, (9 / 100)
val_acc: 0.079000, lr: 5.401602e-06, reg: 1.599828e+04, (10 / 100)
val_acc: 0.154000, lr: 1.618500e-06, reg: 4.925252e-01, (11 / 100)

val_acc: 0.527000, lr: 5.340517e-04, reg: 4.097824e-01, (0 / 100)
val_acc: 0.492000, lr: 2.279484e-04, reg: 9.991345e-04, (1 / 100)
val_acc: 0.512000, lr: 8.680827e-04, reg: 1.349727e-02, (2 / 100)
val_acc: 0.461000, lr: 1.028377e-04, reg: 1.220193e-02, (3 / 100)
val_acc: 0.460000, lr: 1.113730e-04, reg: 5.244309e-02, (4 / 100)
val_acc: 0.490000, lr: 9.477770e-04, reg: 2.001293e-03, (5 / 100)
val_acc: 0.469000, lr: 1.484369e-04, reg: 4.328313e-01, (6 / 100)
val_acc: 0.522000, lr: 5.586261e-04, reg: 2.312685e-04, (7 / 100)
val_acc: 0.530000, lr: 5.800183e-04, reg: 8.259964e-02, (8 / 100)
val_acc: 0.489000, lr: 1.979160e-04, reg: 1.010009e-04, (9 / 100)
val_acc: 0.490000, lr: 2.036031e-04, reg: 2.406271e-03, (10 / 100)
val_acc: 0.475000, lr: 2.021162e-04, reg: 2.287807e-01, (11 / 100)
val_acc: 0.460000, lr: 1.135527e-04, reg: 3.905040e-02, (12 / 100)
val_acc: 0.515000, lr: 6.947668e-04, reg: 1.562808e-02, (13 / 100)
val_acc: 0.531000, lr: 9.471540e-04, reg: 1.433095e-03, (14 / 100)
val_acc: 0.509000, lr: 3.140888e-04, reg: 2.057510e-01, (15 / 100)
val_acc: 0.514000, lr: 6.438349e-04, reg: 3.033781e-01, (16 / 100)
val_acc: 0.502000, lr: 3.921704e-04, reg: 2.707126e-04, (17 / 100)
val_acc: 0.509000, lr: 9.752279e-04, reg: 2.850865e-03, (18 / 100)
val_acc: 0.500000, lr: 2.412040e-04, reg: 4.997021e-04, (19 / 100)
val_acc: 0.466000, lr: 1.319314e-04, reg: 1.189915e-02, (20 / 100)
val_acc: 0.516000, lr: 8.039527e-04, reg: 1.528291e-02, (21 / 100)
```

Fei-Fei Li & Justin Johnson & Serena Yeung

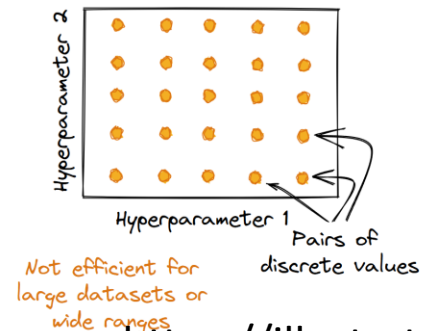
Lecture 7 - 12

April 25, 2017

https://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture7.pdf

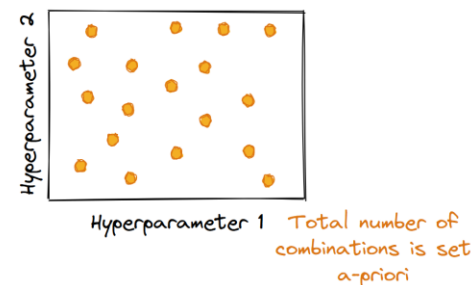
Grid Search

Discretize each of the hyperparameter and evaluate each pair of values



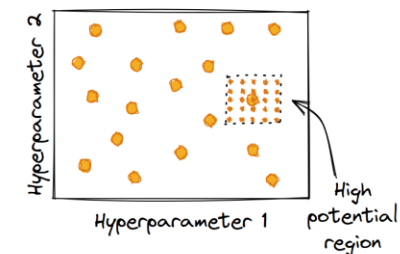
Random Search

Provide a statistical distribution for each hyperparameter from which values are randomly sampled



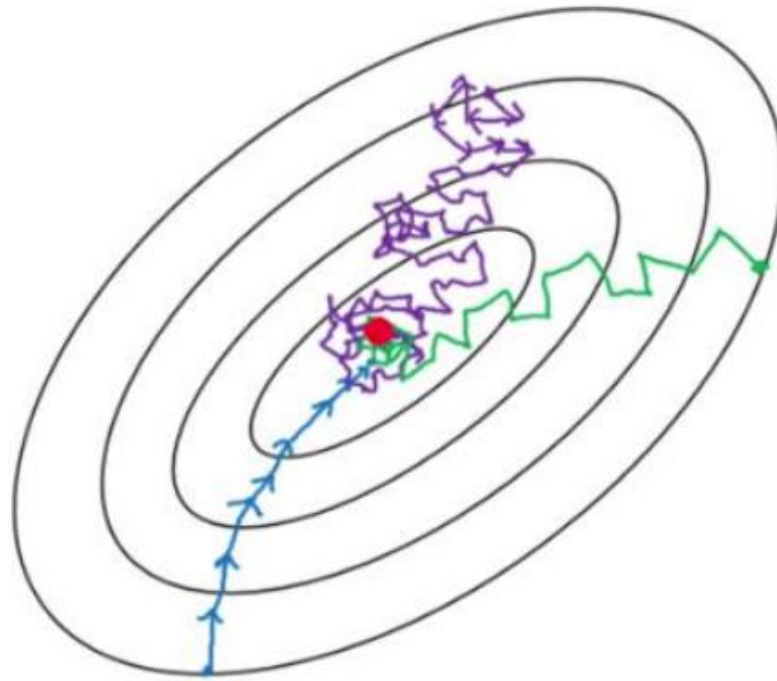
Coarse-to-fine search

Combination of the two. Coarse random search to find regions of high potential, then fine grid search in these regions



<https://illustrated-machine-learning.github.io/#/machine-learning/tuning>

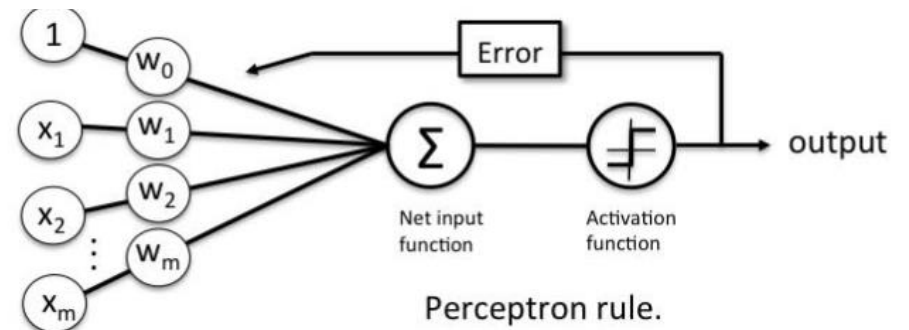
Gradientinio nusileidimo tipai



- Batch gradient descent
- Mini-batch gradient Descent
- Stochastic gradient descent

Neurono mokymas

- Svoriai **pakeičiami pateikus vieną** įėjimo vektorių.
- Mokymo procesas kartojamas **daug kartų pateikiant** visus įėjimo vektorius.
- **Mokymas stabdomas** arba atlikus iš anksto nustatytą **iteracijų (ar epochų) skaičių**, arba pasiekus norimą **mažą paklaidos reikšmę**.



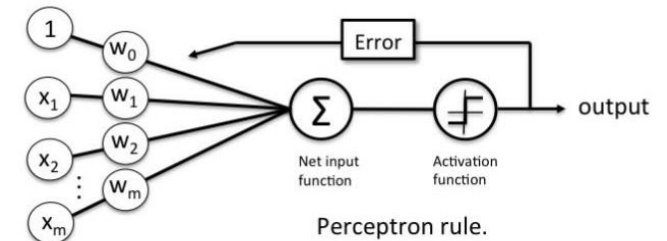
Sigmoidinio neurono mokymo taisyklė

- Tarkime turime **duomenis** $X_i = (x_{i1}, x_{i2}, \dots, x_{in})$, $i = 1, \dots, m$, kuriuos pateiksime į **neurono įėjimus**. Taip pat turime kiekvienam duomenų įrašui **žymes** (label) t_i , kurios sprendžiant klasifikavimo uždavinį atitinka klases.
- Iš pradžių **generuojamos atsitiktinės svorių** w_k , $k = 0, \dots, n$, reikšmės (čia $w_0 = b$).
- Tuomet vykdomas iteracinis procesas, vienos iteracijos metu į neurono įvestį pateikus i -tąjį duomenų įrašą X_i :
 - Suskaičiuojama suma $a_i = \sum_{k=0}^n w_k x_{ik}$.
 - Apskaičiuojamos y_i reikšmė pagal **sigmoidinę funkciją**
 $y_i = f(a_i) = \frac{1}{1+e^{-a_i}}$.
 - Svoriai keičiami (atnaujinami) pagal šią **mokymo taisyklę** (formulę):

$$w_k := w_k - \eta(y_i - t_i)y_i(1 - y_i)(x_{ik})$$

η – yra teigiamas daugiklis, vadinamas **mokymo greičiu** (learning rate).

Čia taikomas **stochastinis gradientinis nusileidimas**.



4.6	3.4	1.4	0.3	Iris-setosa
5.0	3.4	1.5	0.2	Iris-setosa
4.4	2.9	1.4	0.2	Iris-setosa
4.9	3.1	1.5	0.1	Iris-setosa
5.4	3.7	1.5	0.2	Iris-setosa
4.8	3.4	1.6	0.2	Iris-setosa
4.8	3.0	1.4	0.1	Iris-setosa
4.3	3.0	1.1	0.1	Iris-setosa
5.8	4.0	1.2	0.2	Iris-setosa
-	-	-	-	-

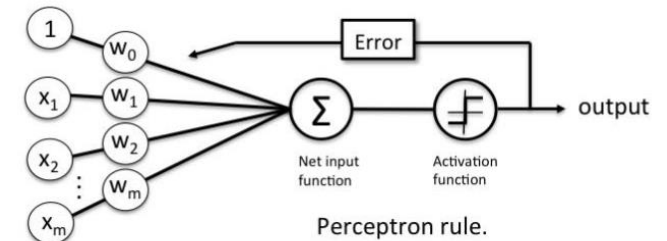
Sigmoidinio neurono mokymo taisyklė

- Reikia **minimizuoti** funkciją $E(W) = \frac{1}{2} \sum_{i=1}^m (t_i - y_i)^2$.
- Tai suma paklaidų **kiekvienam** įėjimų vektoriui:
 $E(W) = \sum_{i=1}^m E_i$.
- Prisiminkime, kad $y_i = f(a_i) = f(\sum_{k=0}^n w_k x_{ik})$.
- Naudojama **sigmoidinė aktyvacijos funkcija**
 $f(a_i) = \frac{1}{1+e^{-a_i}}$. Jos išvestinė $f'(a_i) = f(a_i)(1 - f(a_i))$.
- Tuomet funkcijos $E(W)$ **išvestinė** pagal w_k :

$$\begin{aligned} \frac{\partial E_i(W)}{\partial w_k} &= (t_i - y_i) \times \left(-\frac{\partial f(a_i)}{\partial w_k} \right) = (t_i - y_i)(-f(a_i)(1 - f(a_i))) \\ &= (y_i - t_i)y_i(1 - y_i). \end{aligned}$$

- Svoriai keičiami (atnaujinami) pagal šią **mokymo taisyklę** (formulę):

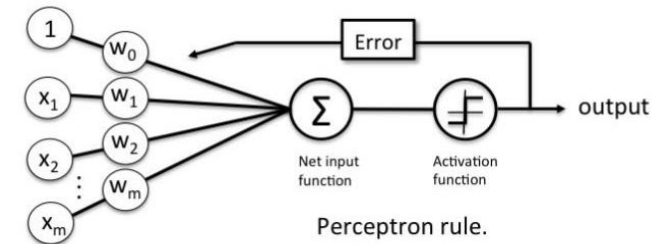
$$w_k := w_k - \eta(y_i - t_i)y_i(1 - y_i)(x_{ik})$$



4.6	3.4	1.4	0.3	Iris-setosa
5.0	3.4	1.5	0.2	Iris-setosa
4.4	2.9	1.4	0.2	Iris-setosa
4.9	3.1	1.5	0.1	Iris-setosa
5.4	3.7	1.5	0.2	Iris-setosa
4.8	3.4	1.6	0.2	Iris-setosa
4.8	3.0	1.4	0.1	Iris-setosa
4.3	3.0	1.1	0.1	Iris-setosa
5.8	4.0	1.2	0.2	Iris-setosa
-	-	-	-	-

Neurono mokymas taikant stochastinį gradientinį nusileidimą

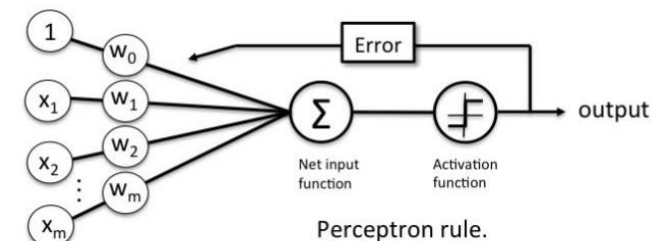
- Nustatome **pradinių svorių** (ir poslinkio) $W = (w_0, w_1, w_2, \dots, w_n)$ reikšmes, **mokymo greitį** η , **epochų skaičių** $epochs$, **paklaidos** siekiamo tikslumo reikšmę E_{min} .
- $totalError = \infty, epoch = 0$
- WHILE** ($totalError > E_{min}$ AND $epoch < epochs$):
//kartojame kol nėra pasiektas norimas tikslumas ir nustatytas epochų skaičius
 - Sumaišome mokymo duomenų įrašus.
 - $totalError = 0$
 - FOR** $i = 1, 2, \dots, m$:
 - FOR** $k = 0, 1, \dots, n$:
 - $w_k := w_k + \eta(t_i - y_i)(x_{ik})$ (*//perceptrono ir ADALINE atveju*)
 - arba $w_k := w_k - \eta(y_i - t_i)y_i(1 - y_i)(x_{ik})$ (*//sigmoidinio neurono atveju*)
 - $error = (t_i - y_i)^2$
 - $totalError = totalError + error$
 - $epoch = epoch + 1$



1	4.6, 3.4, 1.4, 0.3	Iris-setosa
2	5.0, 3.4, 1.5, 0.2	Iris-setosa
3	4.4, 2.9, 1.4, 0.2	Iris-setosa
4	4.9, 3.1, 1.5, 0.1	Iris-setosa
5	5.4, 3.7, 1.5, 0.2	Iris-setosa
6	4.8, 3.4, 1.6, 0.2	Iris-setosa
7	4.8, 3.0, 1.4, 0.1	Iris-setosa
8	4.3, 3.0, 1.1, 0.1	Iris-setosa
9	5.8, 4.0, 1.2, 0.2	Iris-setosa
10	...	...

Sigmoidinio neurono mokymas taikant paketinį gradientinį nusileidimą

- Nustatome **pradinių svorių** (ir poslinkio) $W = (w_0, w_1, w_2, \dots, w_n)$ reikšmes, **mokymo greitį** η , **epochų skaičių** $epochs$, **paklaidos** siekiamo tikslumo reikšmę E_{min} .
- $totalError = \infty, epoch = 0$
- WHILE** ($totalError > E_{min}$ AND $epoch < epochs$):
//kartojame kol nėra pasiektas norimas tikslumas ir nustatytas epochų skaičius
 - Sumaišome mokymo duomenų įrašus.
 - $totalError = 0$
 - $gradientSum = (0, \dots, 0)$
 - FOR** $i = 1, 2, \dots, m$:
 - FOR** $k = 0, 1, \dots, n$:
 - $gradientSum_k = gradientSum_k + (y_i - t_i)y_i(1 - y_i)x_{ik}$
 - $error = (t_i - y_i)^2$
 - $totalError = totalError + error$
 - FOR** $k = 0, 1, \dots, n$:
 - $w_k := w_k - \eta(gradientSum_k/m)$
 - $epoch = epoch + 1$



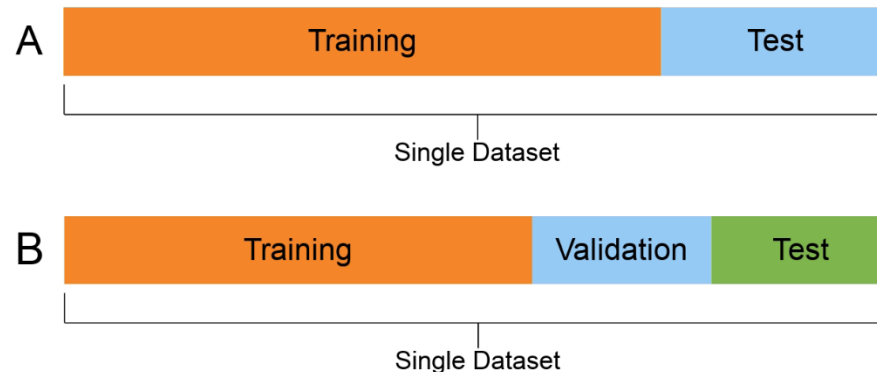
4.6	3.4	1.4	0.3	Iris-setosa
5.0	3.4	1.5	0.2	Iris-setosa
4.4	2.9	1.4	0.2	Iris-setosa
4.9	3.1	1.5	0.1	Iris-setosa
5.4	3.7	1.5	0.2	Iris-setosa
4.8	3.4	1.6	0.2	Iris-setosa
4.8	3.0	1.4	0.1	Iris-setosa
4.3	3.0	1.1	0.1	Iris-setosa
5.8	4.0	1.2	0.2	Iris-setosa
-	-	-	-	-

Programos rezultatas turi būti:

- Gauti neurono svoriai ir poslinkis (*bias*). Kadangi krūties vėžio duomenyse yra 9 požymiai, tai rezultate turi būti 9 svoriai ir vienas poslinkis.
- Gautos paklaidos po kiekvienos epochos mokymo duomenims (žr. <https://emokymai.vu.lt/mod/resource/view.php?id=171951>, 46 skaidrė).
- Gautos paklaidos po kiekvienos epochos validavimo duomenims.
- Gauta paklaida testavimo duomenims (žr. <https://emokymai.vu.lt/mod/resource/view.php?id=171951>, 48 skaidrė).
- Gautas klasifikavimo tikslumas po kiekvienos epochos mokymo ir validavimo duomenims.
- Gautas klasifikavimo tikslumas testavimo duomenims.

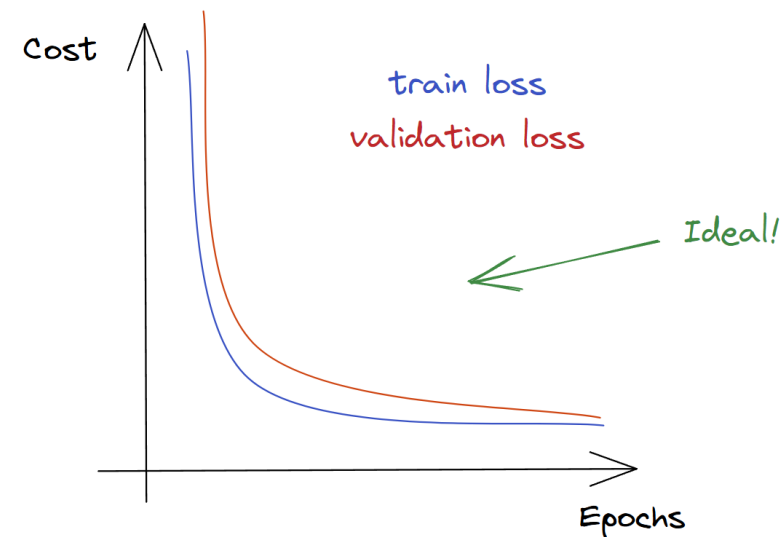
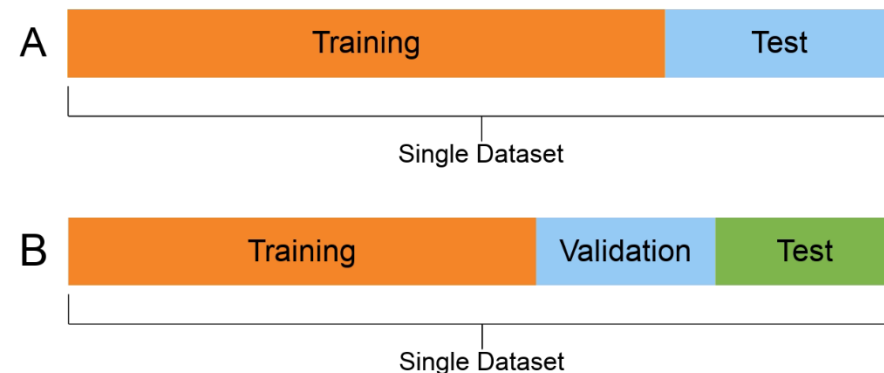
Klasifikavimo tikslumas (*accuracy*) – tai santykis tarp teisingai klasifikuotų ir visų duomenų. Norint jį gauti, reikia kiekvienam duomenų įrašui paskaičiuoti klasę pagal gautus neuronų svorius. Naudojant sigmoidinę funkciją, neurono išėjimo reikšmės yra intervale (0; 1), tad skaičiuojant klasifikavimo tikslumą, šias reikšmes reiktų suapvalinti iki artimiausio sveiko skaičiaus (0 arba 1).

- Apskaičiuojamas neurono mokymosi laikas.



Galutinis paklaidos įvertinimas mokymo duomenims

- Kai **neuronas** yra **išmokytas**, t. y. rasti tinkami svoriai (ir poslinkis), teigiama, kad turime **išmokytą modelį**.
- **Modelio kokybei įvertinti** galime apskaičiuoti **bendrą paklaidą mokymo duomenims**.
- Įvedus **validavimo** ir **testavimo** duomenų sąvokas, paklaida bus vertinama ir šiems duomenims.





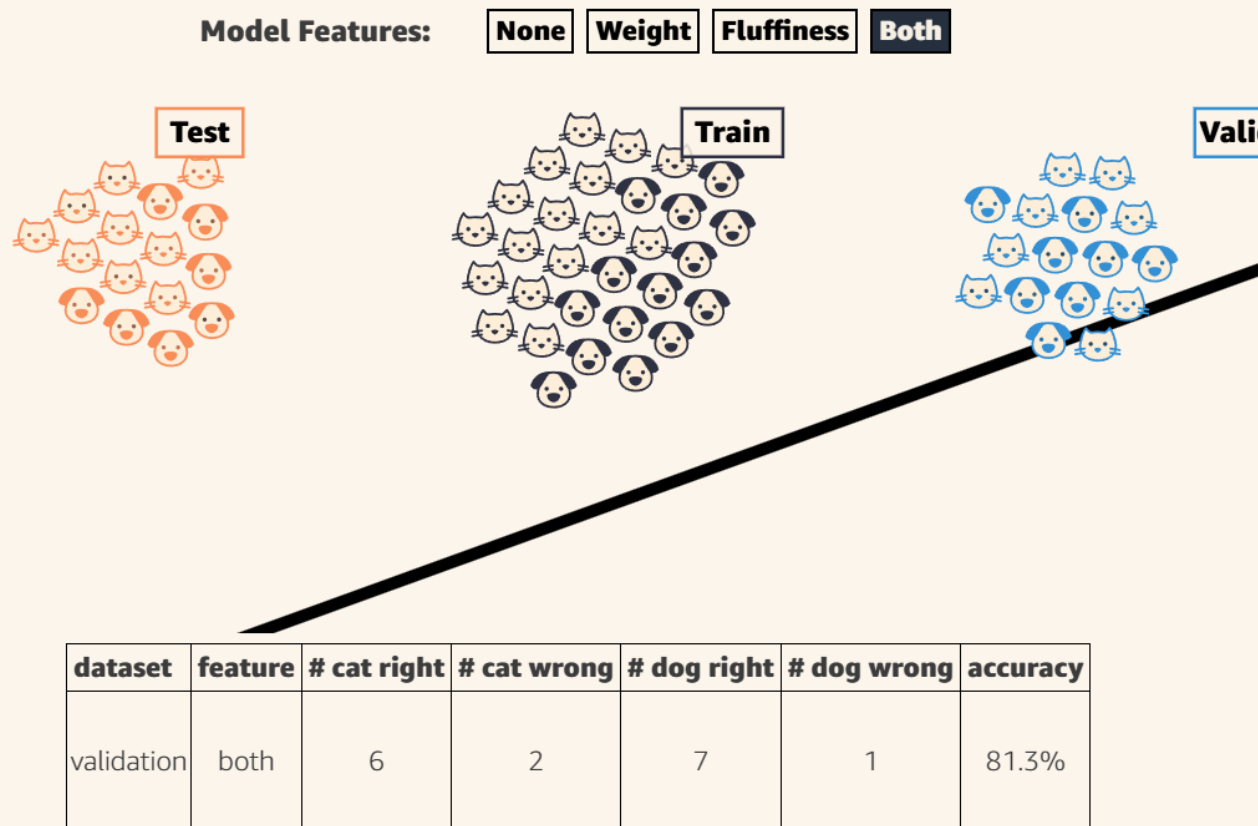
Train, Test, and Validation Splits

The first step in our classification task is to randomly split our pets into three independent sets:

Training Set: The dataset that we feed our model to learn potential underlying patterns and relationships.

Validation Set: The dataset that we use to understand our model's performance across different model types and hyperparameter choices.

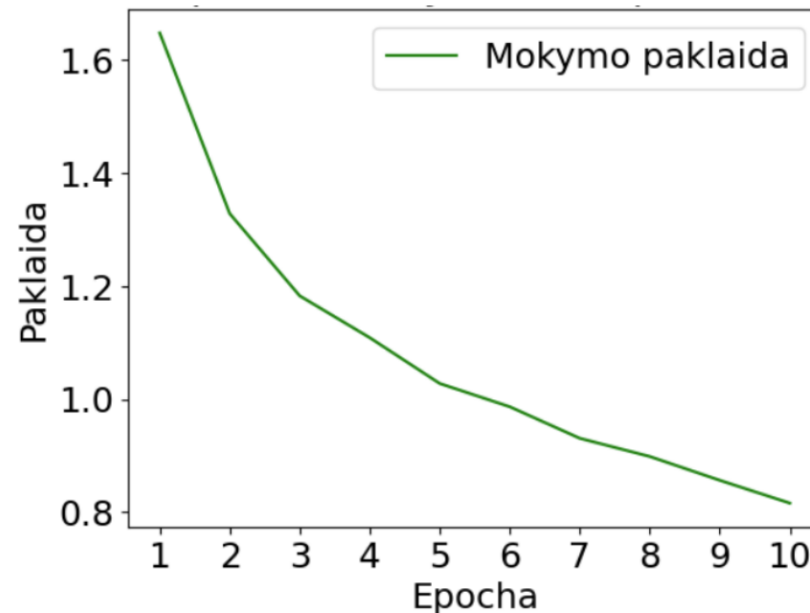
Test Set: The dataset that we use to approximate our model's unbiased accuracy in the wild.



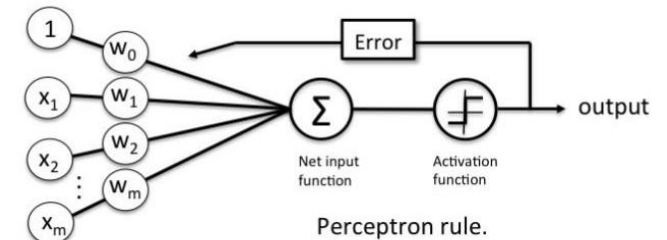
<https://mlu-explain.github.io/train-test-validation/>

Paklaidos priklausomybė nuo epochos numerio

- Mokymo metu verta stebėti **paklaidos priklausomybę nuo epochos** numerio.
- Reikia **kaupiti paklaidos reikšmes**, gautas po kiekvienos epochos.
- Ankstesnėse skaidrėse pateiktuose pseudo koduose reiktų *totalError*, gautos **po kiekvienos epochos, kaupiti į masyvą**.



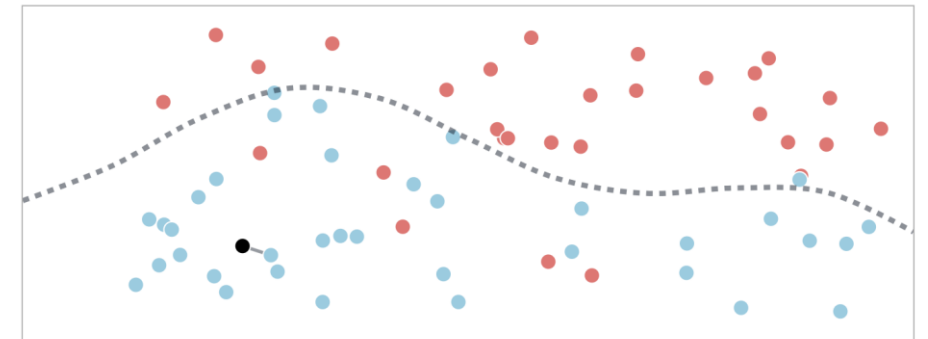
46



```
4.6,3.4,1.4,0.3,Iris-setosa
5.0,3.4,1.5,0.2,Iris-setosa
4.4,2.9,1.4,0.2,Iris-setosa
4.9,3.1,1.5,0.1,Iris-setosa
5.4,3.7,1.5,0.2,Iris-setosa
4.8,3.4,1.6,0.2,Iris-setosa
4.8,3.0,1.4,0.1,Iris-setosa
4.3,3.0,1.1,0.1,Iris-setosa
5.8,4.0,1.2,0.2,Iris-setosa
- - - - -
```


Galutinis paklaidos įvertinimas mokymo duomenims

- Išmokius neuroną, turime svorius $w_0, w_1, w_2, \dots, w_n$ (čia $w_0 = b$).
- **Paklaida** $E(W) = \frac{1}{m} \sum_{i=1}^m (t_i - y_i)^2$, dar vadinama vidutinė kvadratinė paklaida (*mean squared error*, MSE), gali būti skaičiuojama taip:
 - $totalError = 0$
 - **FOR** $i = 1, 2, \dots, m$:
 - $a_i = 0$
 - **FOR** $k = 0, 1, \dots, n$:
 - $a_i = a_i + w_k x_{ik}$
 - $y_i = f(a_i)$
 - $error_i = (t_i - y_i)^2$
 - $totalError = totalError + error_i$
 - $totalError = totalError / m$



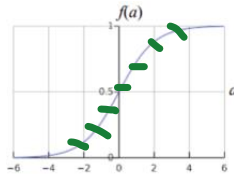
<https://mlu-explain.github.io/bias-variance/>

Klasifikavimo tikslumas (*accuracy*) – tai santykis tarp teisingai klasifikuotų ir visų duomenų. Norint jį gauti, reikia kiekvienam duomenų įrašui paskaičiuoti klasę pagal gautus neuronų svorius. Naudojant sigmoidinę funkciją, neurono išėjimo reikšmės yra intervale (0; 1), tad skaičiuojant klasifikavimo tikslumą, šias reikšmes reiktų suapvalinti iki artimiausio sveiko skaičiaus (0 arba 1).

- Apskaičiuojamas neurono mokymosi laikas.

- Sigmoidinė

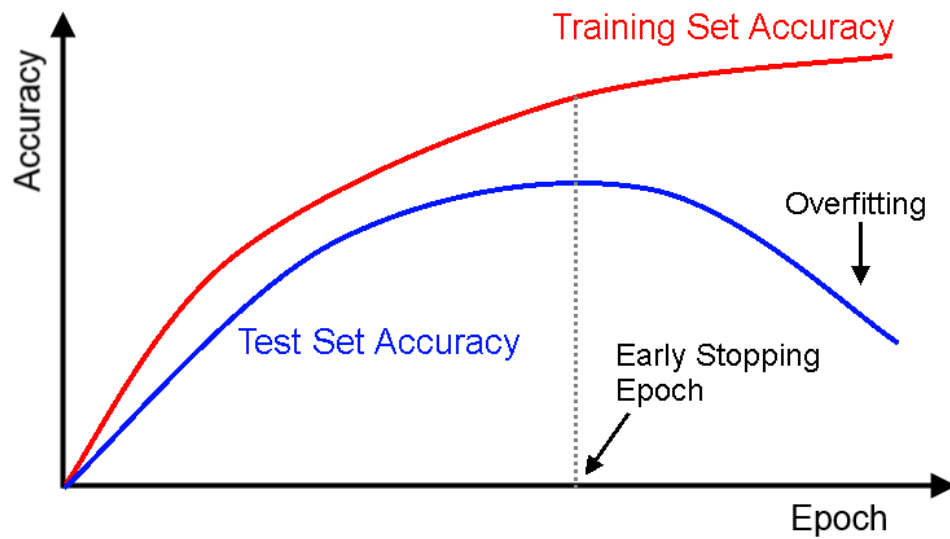
$$f(a) = \frac{1}{1 + e^{-a}}$$



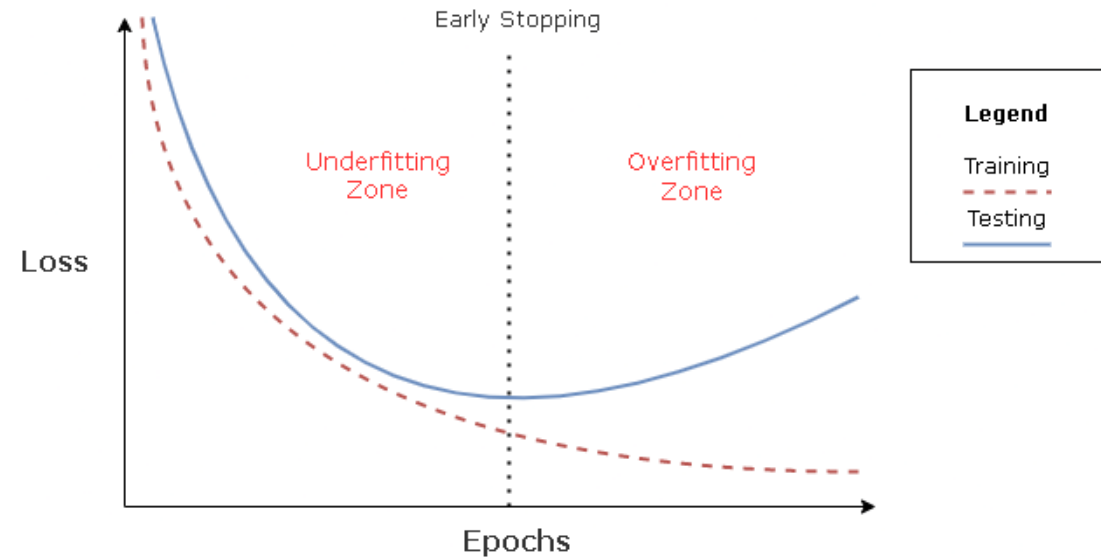
4.6,3.4,1.4,0.3,Iris-setosa
 5.0,3.4,1.5,0.2,Iris-setosa
 4.4,2.9,1.4,0.2,Iris-setosa
 4.9,3.1,1.5,0.1,Iris-setosa
 5.4,3.7,1.5,0.2,Iris-setosa
 4.8,3.4,1.6,0.2,Iris-setosa
 4.8,3.0,1.4,0.1,Iris-setosa
 4.3,3.0,1.1,0.1,Iris-setosa
 5.8,4.0,1.2,0.2,Iris-setosa
 _ _ _ _ _

$$E(W) = \frac{1}{2} \sum_{i=1}^m (t_i - y_i)^2$$

pvz. (0 - 0,77)

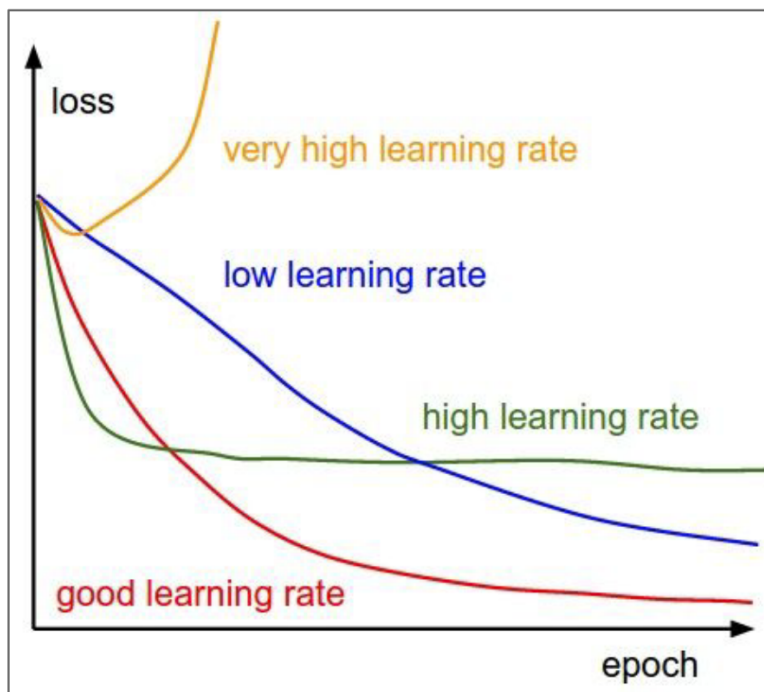


<https://forum.knime.com/t/how-to-generate-accuracy-curve-by-epoch-to-detect-overfitting-with-training-validation-data/11916>

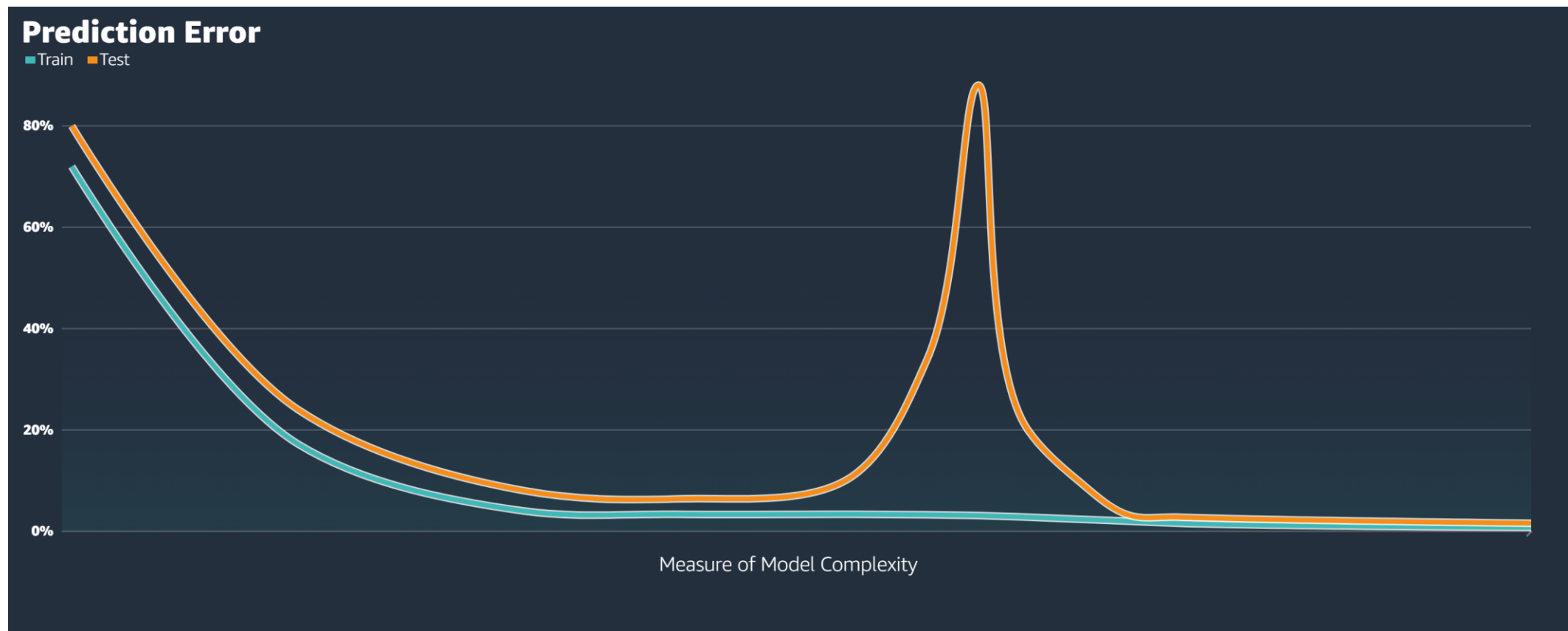


<https://towardsdatascience.com/the-million-dollar-question-when-to-stop-training-deep-learning-models-fa9b488ac04d>

SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have **learning rate** as a hyperparameter.



Q: Which one of these learning rates is best to use?



<https://mlu-explain.github.io/double-descent/>

3. Atlikti tyrimus. Kiekvieno tyrimo rezultatus (klasifikavimo tikslumas ir paklaida) pateikti grafikuose arba lentelėse su atitinkamais komentaramis. Tie rezultatai, kuriuos lengviau ir greičiau galima suprasti iš grafikų, jie turi būti pateikti grafikuose. Pavyzdžiui, paklaidos ir klasifikavimo tikslumo priklausomybė nuo epochų reiktų vaizduoti linijiniu grafiku. Norint pamatyti skirtumus ar panašumus tarp rezultatų, esant įvairioms tam tikro hiperparametro reikšmėms, rezultatus vertėtų pateikti stulpelinėse diagramose. Tinkamas rezultatų vaizdavimas gali įtakoti užduoties įvertinimą.

Tyrimų metu būtina nustatyti ir aprašyti:

- Kaip paklaidos reikšmės priklauso nuo epochų skaičiaus mokymo ir validavimo duomenims? Pateikti grafiką, kurio x ašyje yra atidėdamos epochos, o y ašyje paklaidų reikšmės mokymo ar validavimo duomenims.
- Kaip klasifikavimo tikslumas priklauso nuo epochų skaičiaus mokymo ir validavimo duomenims? Pateikti grafiką, kurio x ašyje yra atidėdamos epochos, o y ašyje klasifikavimo tikslumas mokymo ar validavimo duomenims.
- Kaip rezultatai priklauso nuo skirtingų mokymosi greičio reikšmių? Tinkamą vaizdavimo būdą (lentelės, grafikai) galima pasirinkti patiems. Mokymosi greičio reikšmė turėtų būti intervale (0, 1), eksperimentų metu rekomenduojama panagrinėti atvejus su mažiausiai trimis skirtingomis mokymosi greičio reikšmėmis.
- Kaip rezultatus (paklaidą ir klasifikavimo tikslumą) įtakoja taikomas gradientinis nusileidimas (paketinis ir stochastinis).
- Kaip taikomas gradientinis nusileidimas (paketinis ar stochastinis) įtakoja mokymo laiką esant vienodam epochų skaičiui.

With infinite time we would tune all hyperparameters..

↪ ..but unfortunately we have to prioritize them

Hyperparameter	Sensitivity
Learning Rate	High
Learning Rate Schedule	High
Loss Function	High
Units per Layer	High
Parameter initialization strategy	Medium
Number of layers	Medium
Layer Properties	Medium
Degree of Regularization	Medium
Choice of Optimizer	Low
Optimizer Properties	Low
Size of minibatch	Low
Choice of non-linearity	Low

<https://illustrated-machine-learning.github.io/#/machine-learning/tuning>

Užduoties ataskaitoje:

- Aprašyti, kokie duomenys buvo naudojami, kiek yra duomenų įrašų (eilučių), požymių (stulpelių).
- Aprašyti, kaip duomenys buvo padalinti į mokymo, validavimo ir testavimo aibes. Paaiškinti, kokia kiekvienos aibės paskirtis.
- Paaiškinti, kaip buvo parinktos pradinių svorių reikšmės.
- Pateikti programos tekstą su išsamiais komentarais.
- Paaiškinti, kas yra paketinis gradientinis nusileidimas ir stochastinis gradientinis nusileidimas. Paaiškinti, kas kiekvienu atveju yra epocha.
- Detaliai aprašyti atlikto tyrimo rezultatus (žr. 3 punktą). Aprašykite, kaip rezultatai skiriasi naudojant vieną ar kitą gradientinį nusileidimą.
- Apmokius neuroną ir nustačius atvejį, kada gaunamas didžiausias klasifikavimo tikslumas ir mažiausia paklaida validavimo duomenims, pateikti:
 - gautus svorius,
 - epochų skaičių,
 - paklaidas paskutinėje epochoje mokymo ir validavimo duomenims,
 - klasifikavimo tikslumą paskutinėje epochoje mokymo ir validavimo duomenimis,
 - paklaidą testavimo duomenims,
 - klasifikavimo tikslumą testavimo duomenimis,
 - kiekvienam testavimo duomenų įrašui nurodyti, kokias klases nustatė neuronas ir kokia turėjo būti.

Pastaba: gali būti, kad nepavyks rasti varianto, kad vienu metu klasifikavimo tikslumas būtų didžiausias, o paklaida – mažiausia. Tuomet savo nuožiūra reikia išrinkti variantą, kad klasifikavimo tikslumas būtų kiek galima didžiausias, o paklaida – kiek galima mažiausia.

- Pateikti atliktų tyrimų išvadas. Išvados turi būti aiškos ir lakoniškos, tiksliai atspindinčios gautus rezultatus. Išvadų apimtis – 4–10 sakinių.